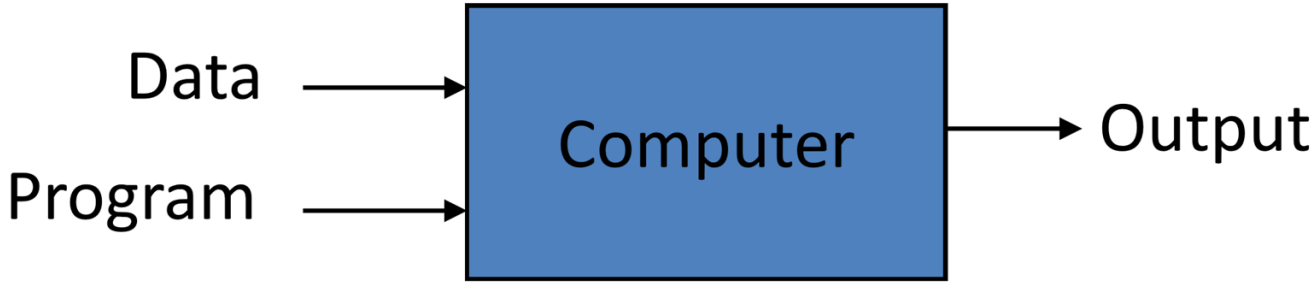
MACHINE LEARNING INTUITION I

DATA/MSML 612: Deep Learning

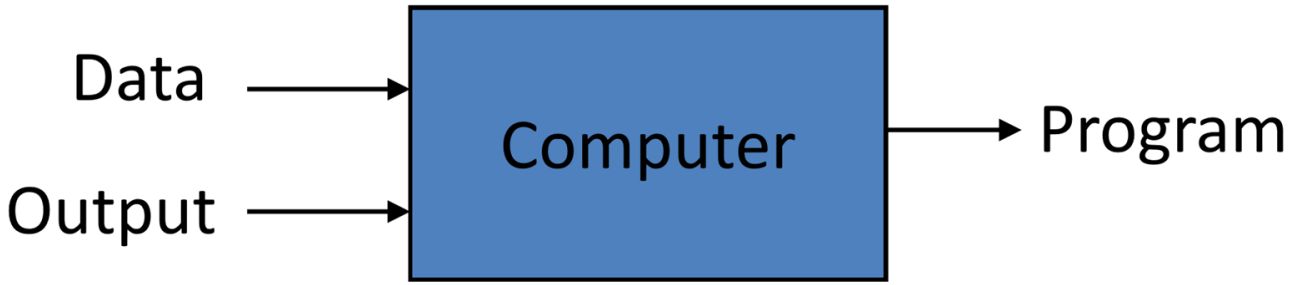
Review: Machine and Deep Learning

Machine Learning (1/9)

- Traditional programming



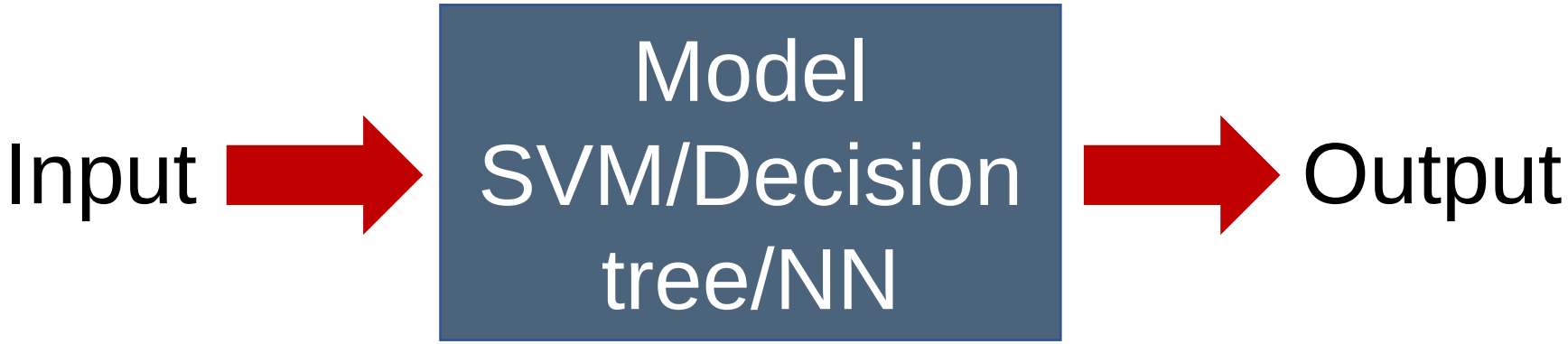
- Machine learning



© 2020, UNIVERSITY OF MARYLAND

Machine Learning (2/9)

- Learn a mapping



© 2020, UNIVERSITY OF MARYLAND

Machine Learning (3/9)

- Two kinds of problem
  - According to the kinds of output ground truth
  - **Classification:** the output ground truth is discretized values (or labels)

Input Model Output

© 2020, UNIVERSITY OF MARYLAND

Machine Learning (4/9)

- Two kinds of problem
  - According to the kinds of output ground truth
  - **Classification:** the output ground truth is discretized values (or labels)
  - **Regression:** the output ground truth is continuous values

Training Data

Age (days)	Weight (grams)
2	49
12	122
8	74
21	205
4	80

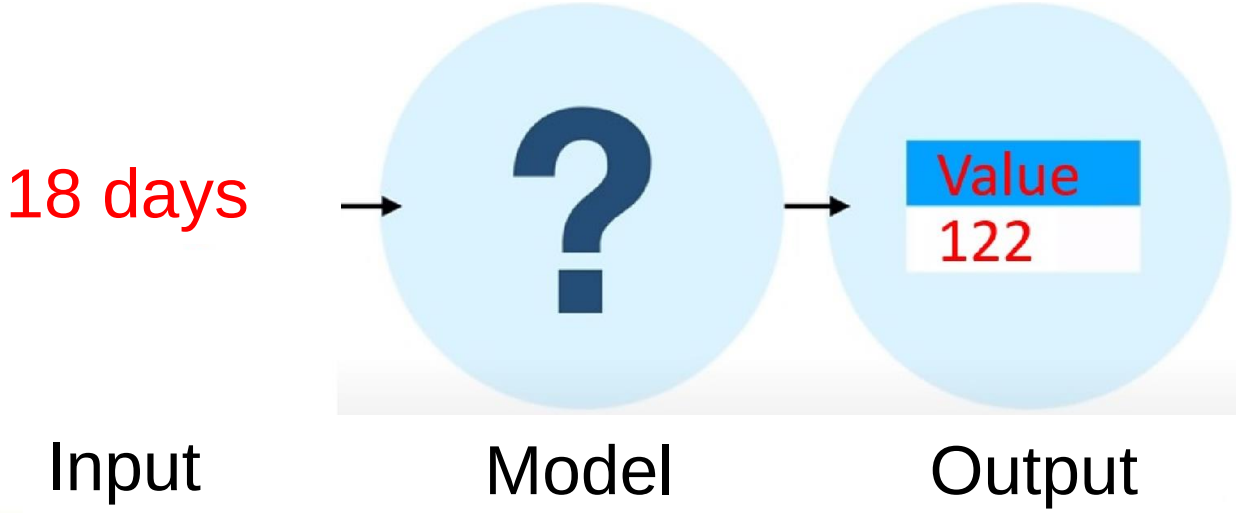
Test Data

Age (days)	Weight (grams)
18	?

© 2020, UNIVERSITY OF MARYLAND

Machine Learning (5/9)

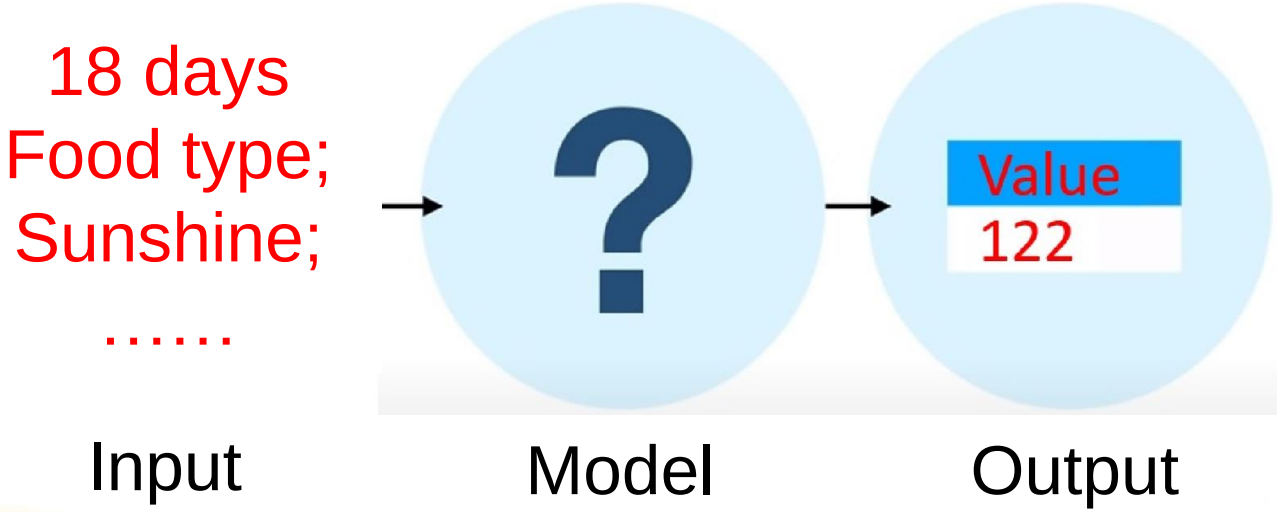
- Two kinds of problem
  - According to the kinds of output ground truth
  - **Classification:** the output ground truth is discretized values (or labels)
  - **Regression:** the output ground truth is continuous values



© 2020, UNIVERSITY OF MARYLAND

Machine Learning (6/9)

- Two kinds of problem
  - According to the kinds of output ground truth
  - **Classification:** the output ground truth is discretized values (or labels)
  - **Regression:** the output ground truth is continuous values

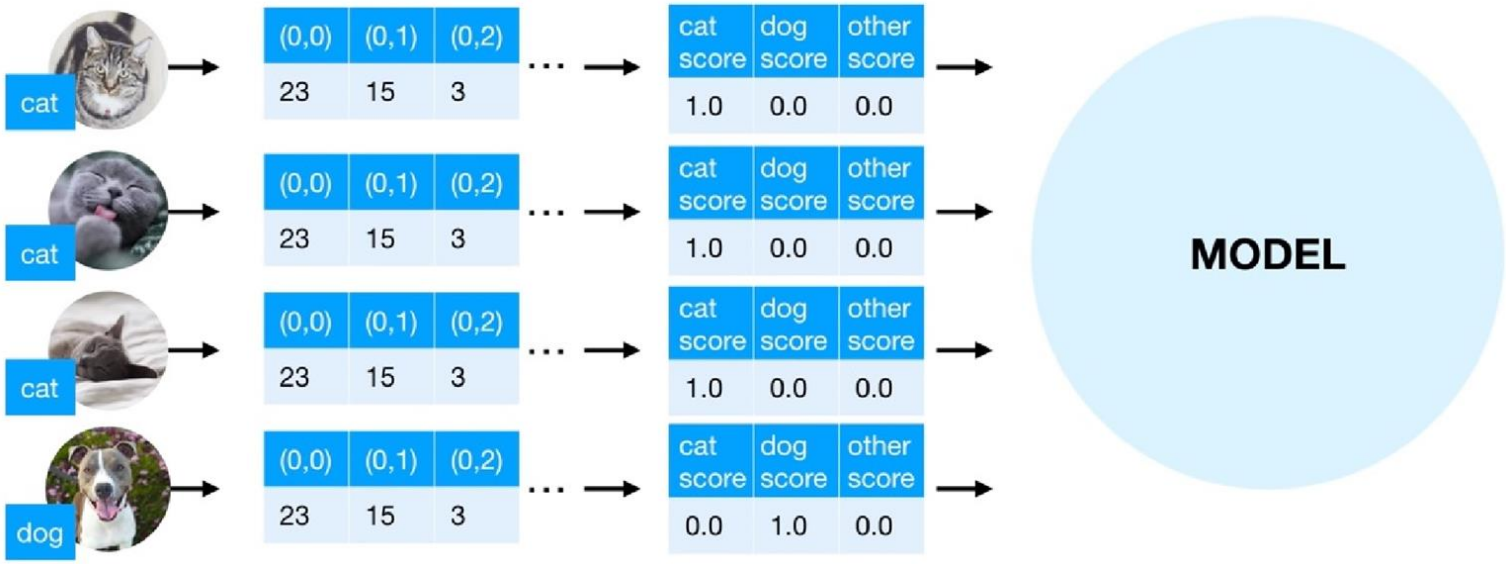


© 2020, UNIVERSITY OF MARYLAND

Machine Learning (7/9)

What should we do to use the models in either classification or regression?

- Two procedures:
  - Training: learn the parameters in a model

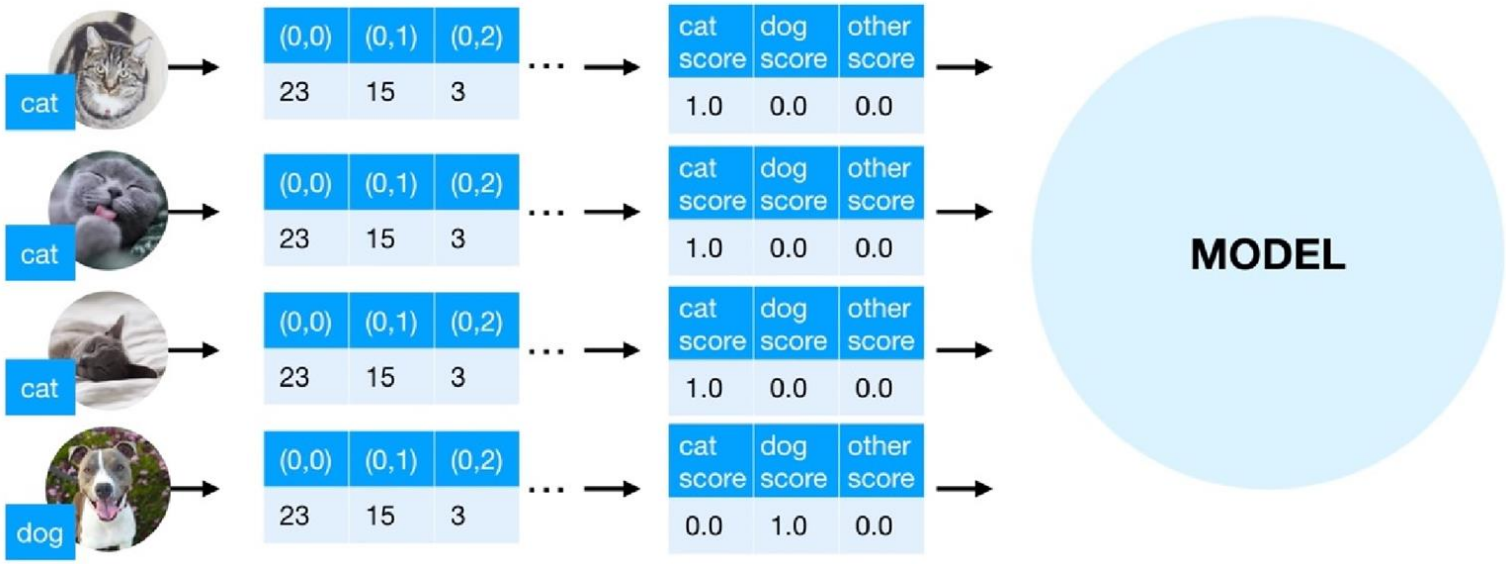


© 2020, UNIVERSITY OF MARYLAND

Machine Learning (8/9)

What should we do to use the models in either classification or regression?

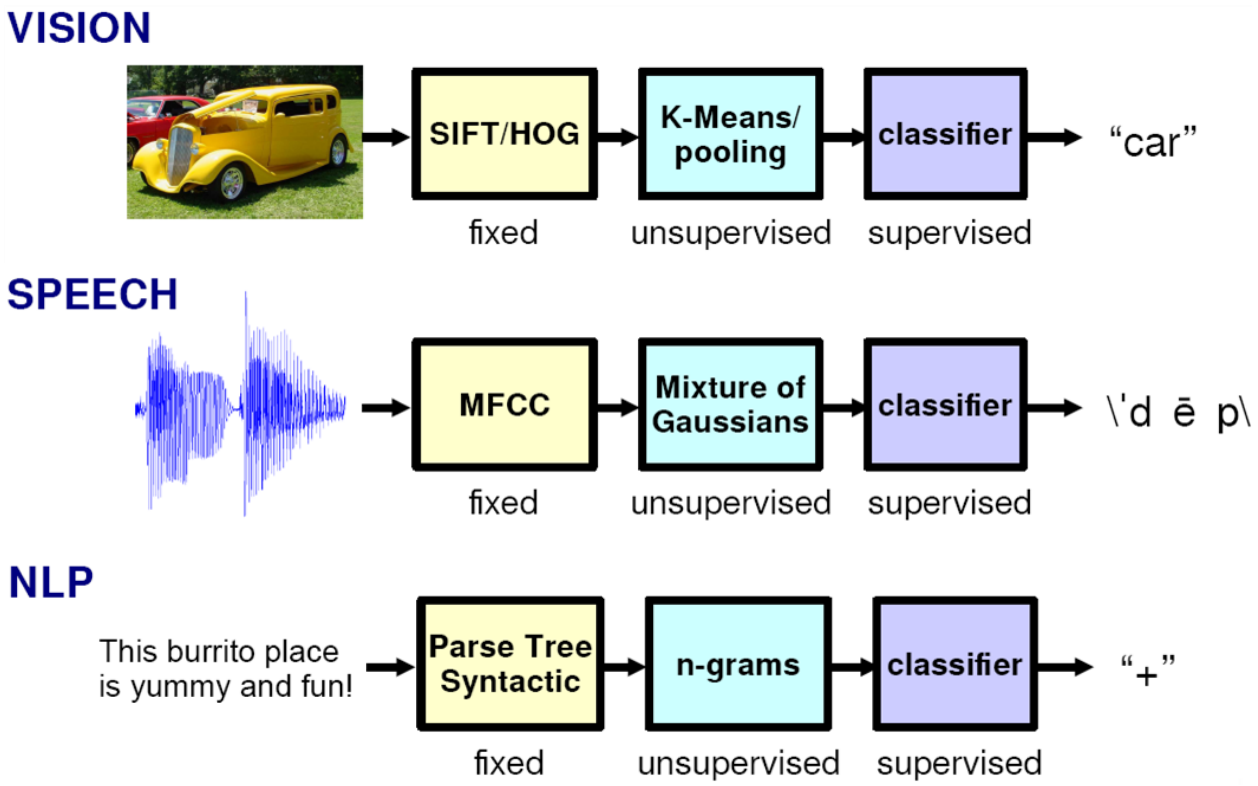
- Two procedures:
  - Training: learn the parameters in a model
  - Testing: use an unseen input to go through the model with the learned parameters



© 2020, UNIVERSITY OF MARYLAND

Machine Learning (9/9)

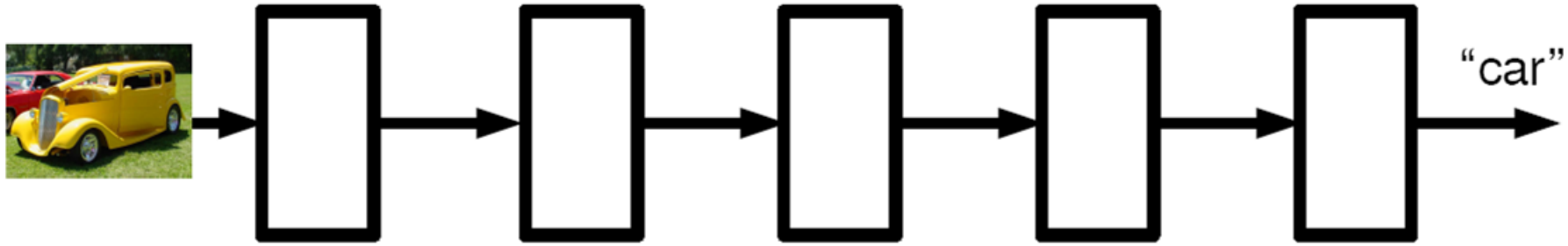
- Traditional pattern recognition



© 2020, UNIVERSITY OF MARYLAND

Deep Learning

- “DL is part of a broader family of machine learning methods based on artificial neural networks with representation learning” – WIKI



- Cascade of non-linear transformations
- End to end learning
- General framework (any hierarchical model is deep)

© 2020, UNIVERSITY OF MARYLAND

Some Math

Some Math (1/4)

- Linear algebra
  - Vector in R^d is an ordered set of d real numbers
    - e.g., v=[1,6,3,4] is in R^4
    - [1,6,3,4] is a column vector: $\begin{pmatrix} 1 \\ 6 \\ 3 \\ 4 \end{pmatrix}$
    - as opposed to a row vector: $\begin{pmatrix} 1 & 6 & 3 & 4 \end{pmatrix}$
  - An m-by-n matrix is an object with m rows and n columns, where each entry is a real number:

$$\begin{pmatrix} 1 & 2 & 8 \\ 4 & 78 & 6 \\ 9 & 3 & 2 \end{pmatrix}$$

© 2020, UNIVERSITY OF MARYLAND

Some Math (2/4)

- Linear algebra
  - Transpose: reflect vector/matrix on line:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^T = \begin{pmatrix} a & c \\ b & d \end{pmatrix} \quad \begin{pmatrix} a \\ b \end{pmatrix}^T = \begin{pmatrix} a & b \end{pmatrix} \quad (Ax)^\top = x^\top A^\top$$

  - Vector norms:
    - Lp norm of **v** = (v1,…,vk) is $\left(\sum_i |v_i|^p \right)^{\frac{1}{p}}$
    - Common norms: L1, L2
    - $L_{infinity}$ or L_{max}= $\max_i$ |vi|
  - Length of a vector v is L2(v)

© 2020, UNIVERSITY OF MARYLAND

Some Math (3/4)

- Linear algebra
  - Vector dot product: $u \bullet v = \begin{pmatrix} u_1 & u_2 \end{pmatrix} \bullet \begin{pmatrix} v_1 & v_2 \end{pmatrix} = u_1 v_1 + u_2 v_2$
  - Dot product of vector with itself: $\text{length}(\boldsymbol{u})^2 = \|\boldsymbol{u}\|_2^2$
  - Matrix product:

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$AB = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

© 2020, UNIVERSITY OF MARYLAND

Some Math (4/4)

- Linear algebra
  - Dot (inner) product: $u \bullet v = u^T v = \begin{pmatrix} u_1 & u_2 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = u_1 v_1 + u_2 v_2$
  - Orthogonal (perpendicular): inner product is 0
  - Outer product: $uv^T = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} \begin{pmatrix} v_1 & v_2 \end{pmatrix} = \begin{pmatrix} u_1 v_1 & u_1 v_2 \\ u_2 v_1 & u_2 v_2 \end{pmatrix}$

© 2020, UNIVERSITY OF MARYLAND

Vectorization (1/2)

- Benefits of vectorization
  - More compact equations
  - Faster code (using optimized matrix libraries)
- Consider our model: $h(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j x_j$
- Let $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \qquad \boldsymbol{x}^\intercal = \begin{bmatrix} 1 & x_1 & \dots & x_d \end{bmatrix}$
- Can write the model in vectorized form as $h(\boldsymbol{x}) = \boldsymbol{\theta}^\intercal \boldsymbol{x}$

© 2020, UNIVERSITY OF MARYLAND

Vectorization (2/2)

- Consider our model for n instances: $h\left(\boldsymbol{x}^{(i)}\right) = \sum_{j=0}^{d} \theta_j x_j^{(i)}$

- Let $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$ $\boldsymbol{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$

$\mathbb{R}^{(d+1)\times 1}$ $\mathbb{R}^{n\times(d+1)}$

- Can write the model in vectorized form as $h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{X}\boldsymbol{\theta}$

© 2020, UNIVERSITY OF MARYLAND

Vectorized vs Non-vectorized

```python
import numpy as np

def unvectorized(n, theta, x):
    prediction = 0.0
    for j in range(n):
        prediction = prediction + theta[j] * x[j]
    return prediction

def vectorized(theta, x):
    prediction = 0.0
    prediction = np.dot(theta.transpose(), x)
    return prediction

if __name__ == '__main__':
    n = 10000
    theta = np.random.rand(n)
    x = np.random.randint(0, 100, n)

    print(unvectorized(n, theta, x))
    print(vectorized(theta, x))
```


© 2020, UNIVERSITY OF MARYLAND

Special Matrices and Vectors

- Unit vector: $||\boldsymbol{x}||_2 = 1$

- Symmetric Matrix: $\boldsymbol{A} = \boldsymbol{A}^\top$

$$A = \begin{bmatrix} 1 & 7 & 3 \\ 7 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}$$

- Orthogonal matrix: $\boldsymbol{A}^\top \boldsymbol{A} = \boldsymbol{A}\boldsymbol{A}^\top = \boldsymbol{I}$ where $\boldsymbol{I}$ is identity matrix

$$\boldsymbol{A}^{-1} = \boldsymbol{A}^\top$$

© 2020, UNIVERSITY OF MARYLAND

Probability Basics (1/6)

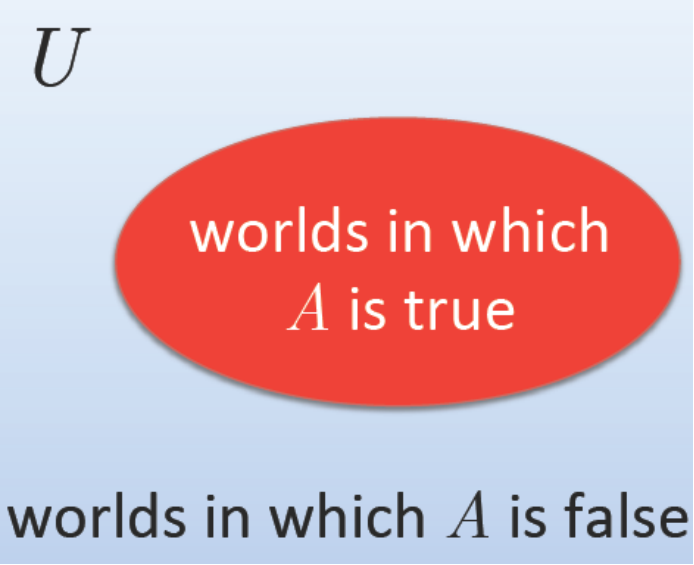
- **Discrete random variables**
  - A represents an event that can take on certain values
  - Each value has an associated probability
- Examples of **binary random variables**:
  - A=I have a headache
  - A=Sally will be the US president in 2020
- **P(A)** is “the fraction of possible worlds in which A is true”

© 2020, UNIVERSITY OF MARYLAND

Probability Basics (2/6)

- Universe U is the event space of all possible worlds
  - Its area is 1
  - P(U)=1
- P(A) = area of red oval
- Therefore: $P(\neg A) = 1 - P(A)$

$$P(A) + P(\neg A) = 1$$



© 2020, UNIVERSITY OF MARYLAND

Axioms of Probability

- All probabilities are between 0 and 1:
  - 0<=P(A)<=1
- Valid propositions (tautologies) have probability 1, and unsatisfiable propositions have probability 0:
  - P(true)=1; P(false)=0
- The probability of a disjunction is given by:
  - P(A∪B)=P(A)+P(B)-P(A∩B)

$$\mathrm{P}(A \vee B) = \mathrm{P}(A) + \mathrm{P}(B) - \mathrm{P}(A \wedge B)$$

© 2020, UNIVERSITY OF MARYLAND

Probability Basics (3/6)

- Interpreting the axioms
  - 0<=P(A)<=1
  - P(true)=1
  - P(false)=0
  - P(A∪B)=P(A)+P(B)-P(A∩B)

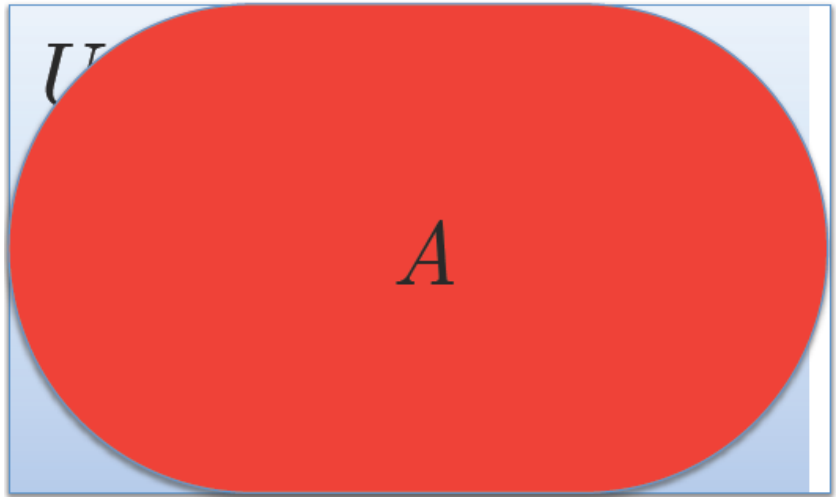
The area of A can't get any smaller than 0

A zero area would mean no world could ever have A true

© 2020, UNIVERSITY OF MARYLAND

Probability Basics (4/6)

- Interpreting the axioms
  - 0<=P(A)<=1
  - P(true)=1
  - P(false)=0
  - P(A∪B)=P(A)+P(B)-P(A∩B)



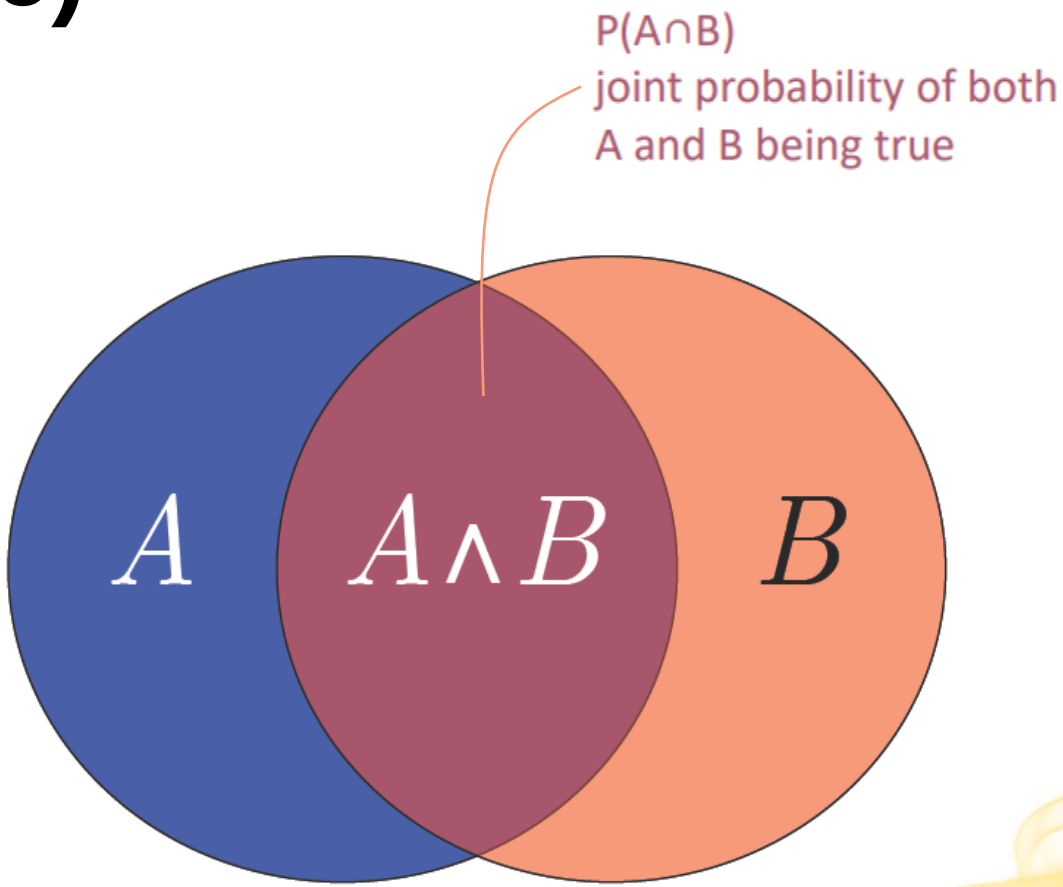
The area of A can't get any any bigger than 1

A area of 1 would mean A is true in all possible worlds

© 2020, UNIVERSITY OF MARYLAND

Probability Basics (5/6)

- Interpreting the axioms
  - 0<=P(A)<=1
  - P(true)=1
  - P(false)=0
  - P(A∪B)=P(A)+P(B)-P(A∩B)



© 2020, UNIVERSITY OF MARYLAND

Probability Basics (6/6)

- Axioms
  - $0 <= P(A) <= 1$
  - $P(true)=1$
  - $P(false)=0$
  - $P(A \cup B)=P(A)+P(B)-P(A \cap B)$
- An important theorem
  - $P(\neg A)=1-P(A)$

© 2020, UNIVERSITY OF MARYLAND

Prior and Joint Probability

- **Prior probability**: degree of belief without any other evidence
- **Joint probability**: matrix of combined probabilities of a set of variables
- Examples:
  - The joint probability is given by:

P(Alarm, Burglary) =

	alarm	¬alarm
burglary	0.09	0.01
¬burglary	0.1	0.8

Prior probability of burglary:

P(Burglary) = 0.1

by marginalization over Alarm

© 2020, UNIVERSITY OF MARYLAND

The Joint Distribution (1/4)

- Recipe for making a joint distribution of d variables:

e.g., Boolean variables A, B, C

© 2020, UNIVERSITY OF MARYLAND

The Joint Distribution (2/4)

- Recipe for making a joint distribution of d variables:
  - Make a truth table listing all combinations of values of your variables (if there are d Boolean variables then the table will have 2^d rows)

e.g., Boolean variables A, B, C

A	B	C	Prob
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

© 2020, UNIVERSITY OF MARYLAND

The Joint Distribution (3/4)

- Recipe for making a joint distribution of d variables
  - Make a truth table listing all combinations of values of your variables (if there are d Boolean variables then the table will have 2^d rows)
  - For each combination of values, say how probable it is

e.g., Boolean variables A, B, C

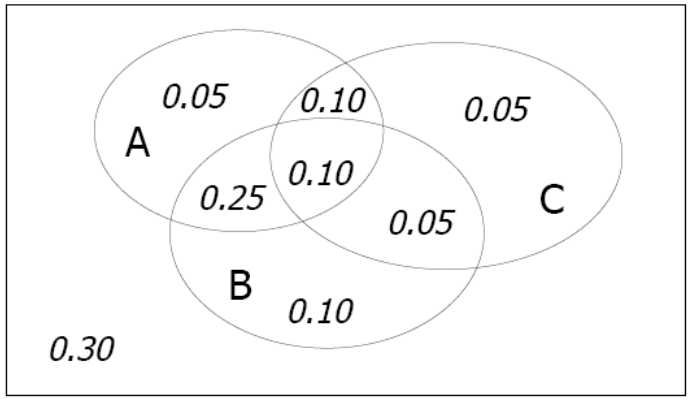
A	B	C	Prob
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

© 2020, UNIVERSITY OF MARYLAND

The Joint Distribution (4/4)

e.g., Boolean variables A, B, C

A	B	C	Prob
0	0	0	0.30
0	0	1	0.05
0	1	0	0.10
0	1	1	0.05
1	0	0	0.05
1	0	1	0.10
1	1	0	0.25
1	1	1	0.10



- Recipe for making a joint distribution of d variables
  - Make a truth table listing all combinations of values of your variables (if there are d Boolean variables then the table will have 2^d rows) .
  - For each combination of values, say how probable it is.
  - If you subscribe to the axioms of probability, all those numbers must sum to 1.

© 2020, UNIVERSITY OF MARYLAND

Inferring Prior Probabilities from the Joint

	alarm		¬alarm	
	earthquake	¬earthquake	earthquake	¬earthquake
burglary	0.01	0.08	0.001	0.009
¬burglary	0.01	0.09	0.01	0.79

$$P(alarm) = \sum_{b,e} P(alarm \wedge \text{Burglary} = b \wedge \text{Earthquake} = e)$$

$$= 0.01 + 0.08 + 0.01 + 0.09 = 0.19$$

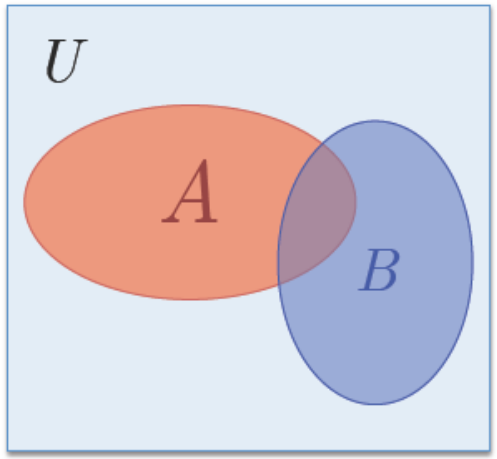
$$P(burglary) = \sum_{a,e} P(\text{Alarm} = a \wedge burglary \wedge \text{Earthquake} = e)$$

$$= 0.01 + 0.08 + 0.001 + 0.009 = 0.1$$

© 2020, UNIVERSITY OF MARYLAND

Conditional Probability (1/6)

- **P(A|B)**= Fraction of worlds in which B is true that also have A true
- What if we already know that B is true?
- That knowledge changes the probability of A
  - Because we know we are in a world where B is true



$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$$

$$P(A \wedge B) = P(A \mid B) \times P(B)$$

© 2020, UNIVERSITY OF MARYLAND

Conditional Probability (2/6)

- Example
  - Explicitly computing priors

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$$

$$P(A \wedge B) = P(A \mid B) \times P(B)$$

P(Alarm, Burglary) =

	alarm	¬alarm
burglary	0.09	0.01
¬burglary	0.1	0.8

$$\begin{aligned} P(\text{burglary} \mid \text{alarm}) &= P(\text{burglary} \wedge \text{alarm}) \;/\; P(\text{alarm}) \\ &= 0.09 \;/\; 0.19 = 0.47 \end{aligned}$$

$$\begin{aligned} P(\text{alarm} \mid \text{burglary}) &= P(\text{burglary} \wedge \text{alarm}) \;/\; P(\text{burglary}) \\ &= 0.09 \;/\; 0.1 = 0.9 \end{aligned}$$

$$\begin{aligned} P(\text{burglary} \wedge \text{alarm}) &= P(\text{burglary} \mid \text{alarm})\, P(\text{alarm}) \\ &= 0.47 * 0.19 = 0.09 \end{aligned}$$

© 2020, UNIVERSITY OF MARYLAND

Conditional Probability (3/6)

- Example
  - Without explicitly computing priors

	alarm		¬alarm	
	earthquake	¬earthquake	earthquake	¬earthquake
burglary	0.01	0.08	0.001	0.009
¬burglary	0.01	0.09	0.01	0.79

$$
\begin{aligned}
P(\text{Burglary} \mid \text{alarm}) &= \alpha\, P(\text{Burglary}, \text{alarm}) \\
&= \alpha\, [P(\text{Burglary}, \text{alarm}, \text{earthquake}) + P(\text{Burglary}, \text{alarm}, \neg\text{earthquake}) \\
&= \alpha\, [\ (0.01, 0.01) + (0.08, 0.09)\ ] \\
&= \alpha\, [\ (0.09, 0.1)\ ]
\end{aligned}
$$

Note: (d_1, d_2) represents a prob. distribution
Burglary = true Burglary = false

Since $P(\text{burglary} \mid \text{alarm}) + P(\neg\text{burglary} \mid \text{alarm}) = 1$,
It must be that $\alpha = 1/(0.09+0.1) = 5.26$ (i.e., $P(\text{alarm}) = 1/\alpha = 0.19$)

$$P(\text{burglary} \mid \text{alarm}) = 0.09 * 5.26 = 0.474$$

$$P(\neg\text{burglary} \mid \text{alarm}) = 0.1 * 5.26 = 0.526$$

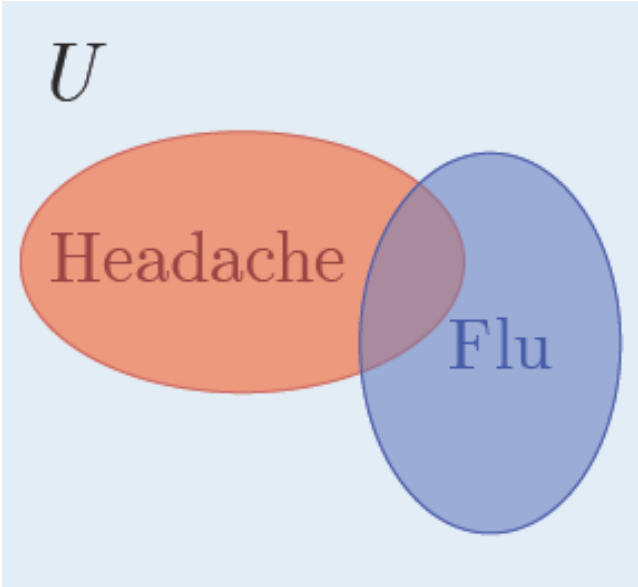
© 2020, UNIVERSITY OF MARYLAND

Conditional Probability (4/6)

- Example
  - Inference from conditional probability

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$$

$$P(A \wedge B) = P(A \mid B) \times P(B)$$



P(headache) = 1/10
P(flu) = 1/40
P(headache | flu) = 1/2

- Headaches are rare and flu is rarer, but if you are coming down with the flu there is a 50-50 chance you will have a headache.

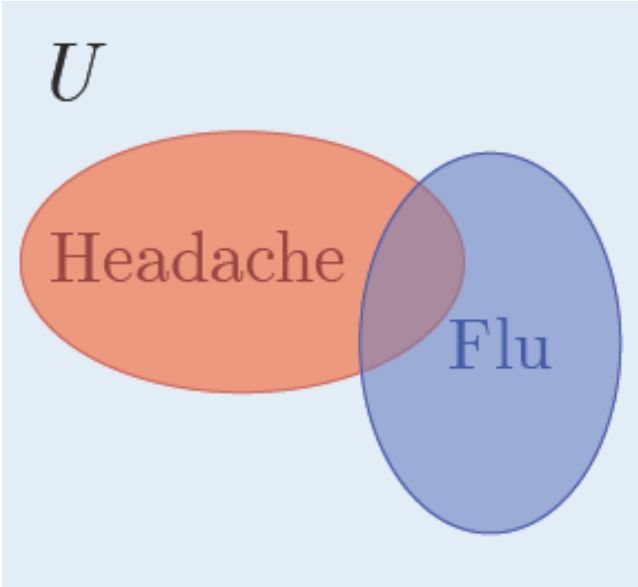
© 2020, UNIVERSITY OF MARYLAND

Conditional Probability (5/6)

- Example
  - Inference from conditional probability

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$$

$$P(A \wedge B) = P(A \mid B) \times P(B)$$



P(headache) = 1/10
P(flu) = 1/40
P(headache | flu) = 1/2

- One day you wake up with a headache. You think "Drat, 50% of flus are associated with headaches so I must have a 50-50 chance of coming down with flu."
  Is this reasoning good?

© 2020, UNIVERSITY OF MARYLAND

Conditional Probability (6/6)

- Example
  - Inference from conditional probability
  - Want to solve for:

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$$

$$P(A \wedge B) = P(A \mid B) \times P(B)$$

P(headache) = 1/10
P(flu) = 1/40
P(headache | flu) = 1/2

P(headache ∧ flu) = ?
P(flu | headache) = ?

P(headache ∧ flu) = P(headache | flu) x P(flu)
= 1/2 x 1/40 = 0.0125

P(flu | headache) = P(headache ∧ flu) / P(headache)
= 0.0125 / 0.1 = 0.125

© 2020, UNIVERSITY OF MARYLAND

Bayes' Rule (1/5)

- Exactly the process we just used
- The most important formula in probabilistic machine learning

(Super Easy) Derivation:

$$\boxed{P(A \wedge B)} = P(A \mid B) \times P(B)$$

$$\boxed{P(B \wedge A)} = P(B \mid A) \times P(A)$$

these are the same

Just set equal...

$$P(A \mid B) \times P(B) = P(B \mid A) \times P(A)$$

and solve...

Bayes, Thomas (1763) An essay towards solving a problem in the doctrine of chances. *Philosophical Transactions of the Royal Society of London*, **53:370-418**

© 2020, UNIVERSITY OF MARYLAND

Bayes' Rule (2/5)

- Allow us to reason from **evidence** to **hypotheses**
- Another way of thinking about Bayes' rule:

$$P(\text{hypothesis} \mid \text{evidence}) = \frac{P(\text{evidence} \mid \text{hypothesis}) \times P(\text{hypothesis})}{P(\text{evidence})}$$

In the flu example:

P(headache) = 1/10 P(flu) = 1/40

P(headache | flu) = 1/2

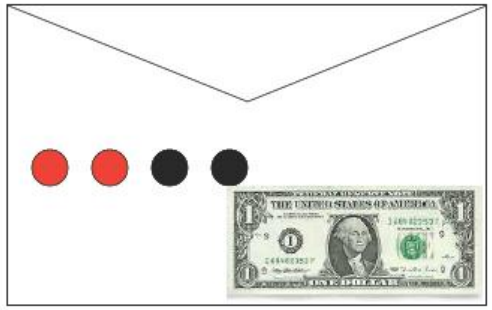
Given evidence of headache, what is P(flu | headache) ?

Solve via Bayes rule!

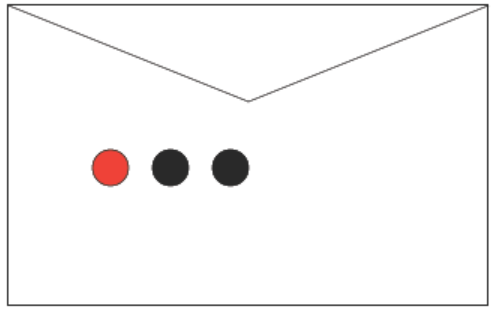
© 2020, UNIVERSITY OF MARYLAND

Bayes' Rule (3/5)

- Using Bayes rule to gamble



The win envelope has a dollar and four beads in it



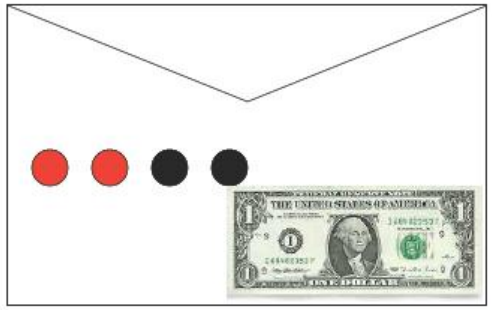
The lose envelope has three beads but no money

Trivial question: Someone draws an envelope at random and offers to sell it to you. How much should you pay?

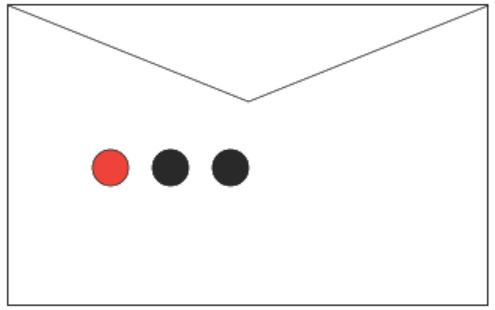
© 2020, UNIVERSITY OF MARYLAND

Bayes' Rule (4/5)

- Using Bayes rule to gamble



The win envelope has a dollar and four beads in it

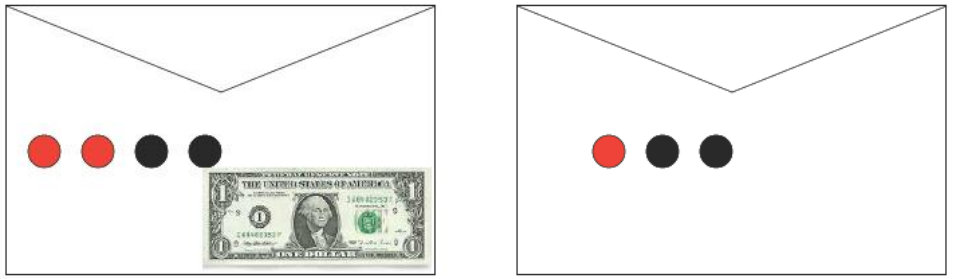


The lose envelope has three beads but no money

Interesting question: Before deciding, you are allowed to see one bead drawn from the envelope.
Suppose it is black: How much should you pay?
Suppose it is red: How much should you pay?

© 2020, UNIVERSITY OF MARYLAND

Bayes' Rule (5/5)



- Calculation:
  - Suppose it is black: How much should you pay?

$$P(b \mid win) = 1/2 \qquad P(b \mid lose) = 2/3$$
$$P(win) = 1/2$$

$$P(win \mid b) = \alpha\, P(b \mid win)\, P(win)$$
$$= \alpha\, 1/2 \times 1/2 = 0.25\alpha$$

$$P(lose \mid b) = \alpha\, P(b \mid lose)\, P(lose)$$
$$= \alpha\, 2/3 \times 1/2 = 0.3333\alpha$$

$$1 = P(win \mid b) + P(lose \mid b) = 0.25\alpha + 0.3333\alpha \Rightarrow \alpha = 1.714$$

$$P(win \mid b) = 0.4286 \qquad P(lose \mid b) = 0.5714$$

© 2020, UNIVERSITY OF MARYLAND

Independence (1/3)

- When two sets of propositions do not affect each others' probabilities, we call them **independent**
- Formal definition:

$$
\begin{aligned}
A \perp\!\!\!\perp B \quad &\leftrightarrow \quad P(A \wedge B) = P(A) \times P(B) \\
&\leftrightarrow \quad P(A \mid B) = P(A)
\end{aligned}
$$

© 2020, UNIVERSITY OF MARYLAND

Independence (2/3)

$$A \perp\!\!\!\perp B \quad \leftrightarrow \quad P(A \wedge B) = P(A) \times P(B)$$
$$\leftrightarrow \quad P(A \mid B) = P(A)$$

- Example

P(smart ∧ study ∧ prep)	smart		¬smart	
	study	¬study	study	¬study
prepared	0.432	0.16	0.084	0.008
¬prepared	0.048	0.16	0.036	0.072

- Is smart independent of study?

© 2020, UNIVERSITY OF MARYLAND

Independence (3/3)

$$A \perp\!\!\!\perp B \quad \leftrightarrow \quad P(A \wedge B) = P(A) \times P(B)$$
$$\leftrightarrow \quad P(A \mid B) = P(A)$$

- Is smart independent of study?

P(smart ∧ study ∧ prep)	smart		¬smart	
	study	¬study	study	¬study
prepared	0.432	0.16	0.084	0.008
¬prepared	0.048	0.16	0.036	0.072

$$\mathrm{P}(\text{study} \wedge \text{smart}) = 0.432 + 0.048 = \boxed{0.48}$$
$$\mathrm{P}(\text{study}) = 0.432 + 0.048 + 0.084 + 0.036 = 0.6$$
$$\mathrm{P}(\text{smart}) = 0.432 + 0.048 + 0.16 + 0.16 = 0.8$$
$$\mathrm{P}(\text{study}) \times \mathrm{P}(\text{smart}) = 0.6 \times 0.8 = \boxed{0.48}$$

So yes!

© 2020, UNIVERSITY OF MARYLAND

Conditional Independence (1/2)

- Absolute of independence of A and B:

$$\begin{aligned} A \perp\!\!\!\perp B \quad &\leftrightarrow \quad P(A \wedge B) = P(A) \times P(B) \\ &\leftrightarrow \quad P(A \mid B) = P(A) \end{aligned}$$

- Conditional independence of A and B given C:

$$A \perp\!\!\!\perp B \mid C \quad \leftrightarrow \quad P(A \wedge B \mid C) = P(A \mid C) \times P(B \mid C)$$

© 2020, UNIVERSITY OF MARYLAND

Conditional Independence (2/2)

- This lets us decompose the joint distribution:
  - Conditional independence is weaker than absolute independence, but still useful in decomposing the full joint

$$P(A \wedge B \wedge C) = P(A \mid C) \times P(B \mid C) \times P(C)$$

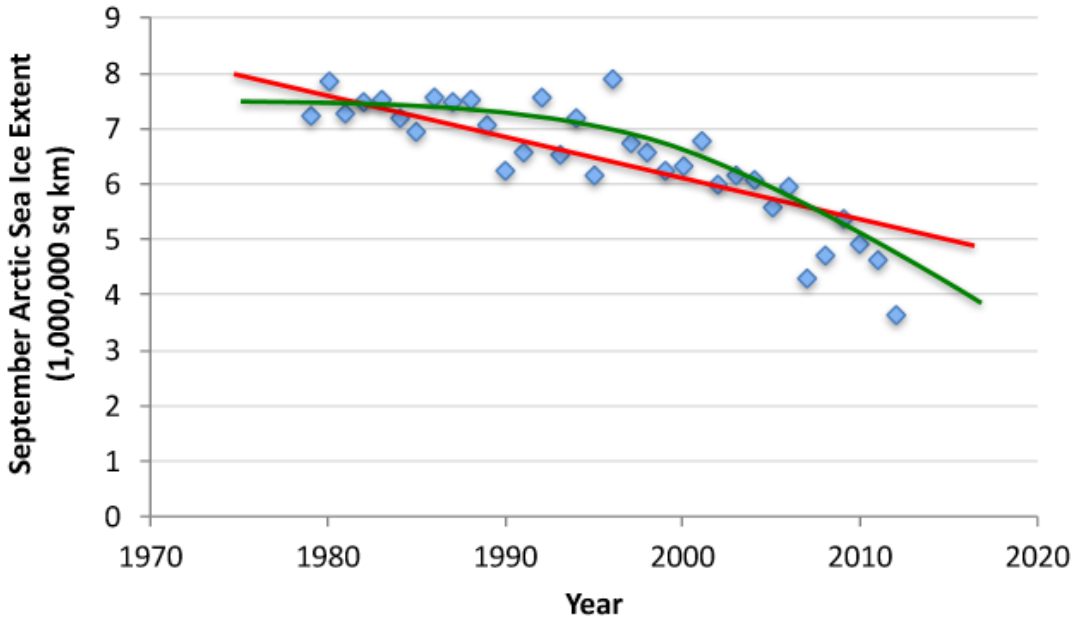
© 2020, UNIVERSITY OF MARYLAND

Some Concepts and Classic Models

Linear Regression (1/2)

- **Regression**
  - Supervised learning: Given training data + desired output (labels)
  - Given (x1, y1), (x2, y2), (x3, y3), …, (xn, yn)
  - Learn a function f(x) to predict y given x

  -- y is real-valued == regression



© 2020, UNIVERSITY OF MARYLAND

Linear Regression (2/2)

- Hypothesis:

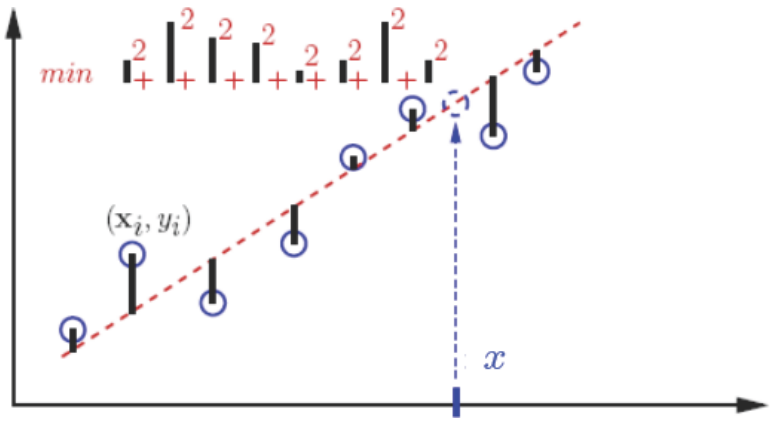
$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_d x_d = \sum_{j=0}^{d} \theta_j x_j$$

Assume $x_0 = 1$

- Fit model by minimizing sum of squared errors

Least squares

The fitted line is used as a predictor



© 2020, UNIVERSITY OF MARYLAND

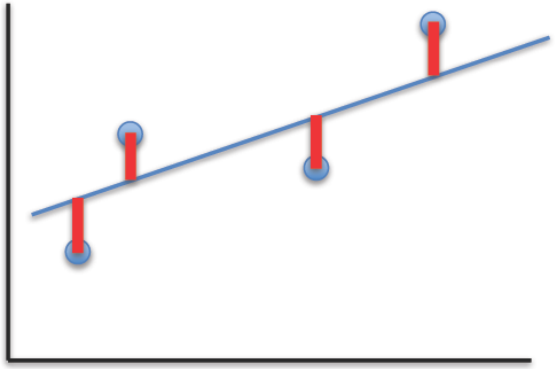
Least Squares Linear Regression

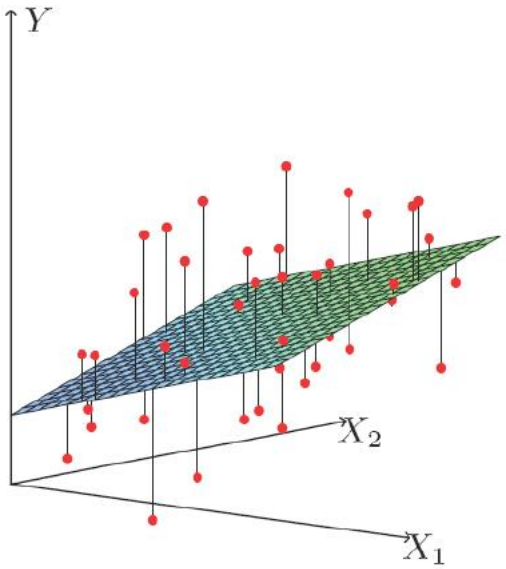
- Cost function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right)^2$$

- Fit by solving

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$





© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (1/9)

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right)^2$$

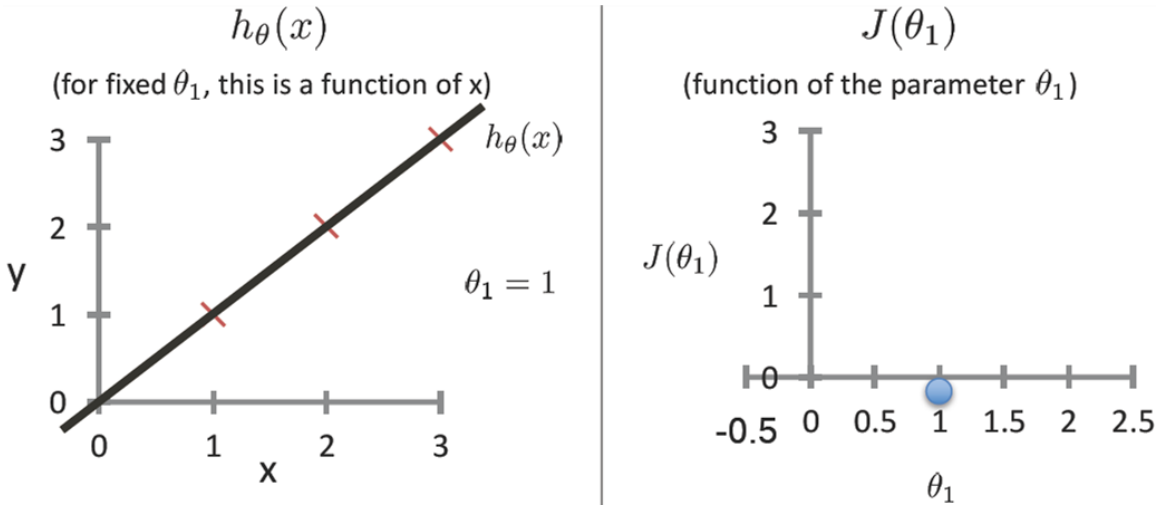
- For insight on J(), let's assume x∈R, $\boldsymbol{\theta} = [\theta_0, \theta_1]$

© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (2/9)

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

- For insight on J(), let's assume x∈R, $\boldsymbol{\theta} = [\theta_0, \theta_1]$

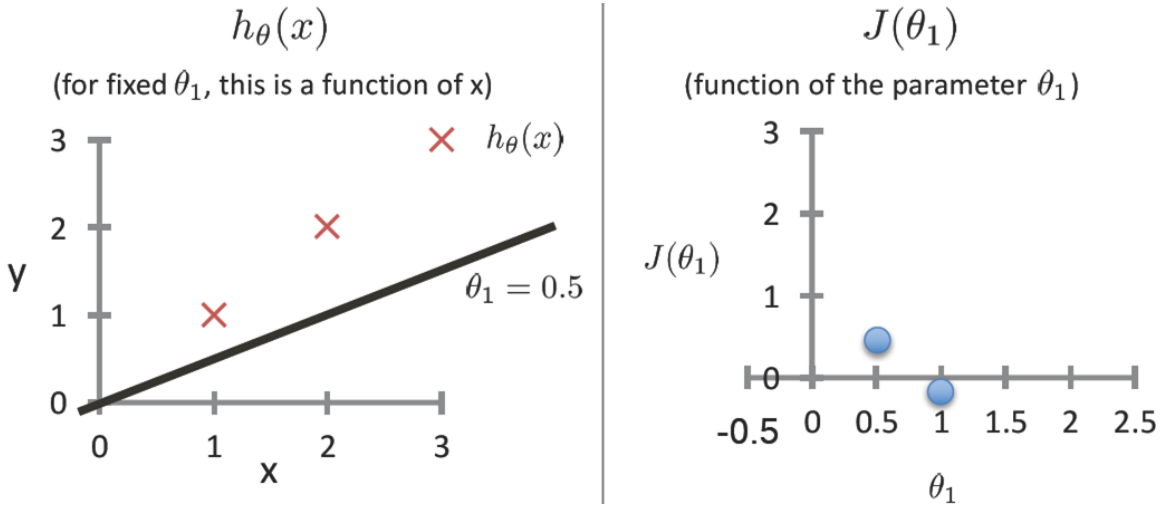


© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (3/9)

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

- For insight on J(), let's assume x∈R, $\boldsymbol{\theta} = [\theta_0, \theta_1]$



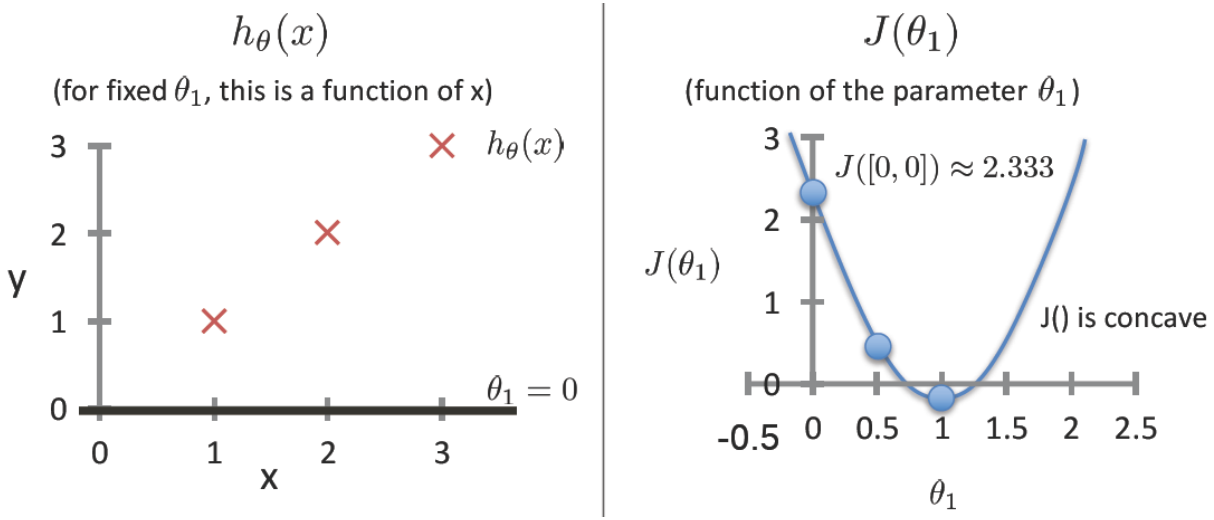
$$J([0, 0.5]) = \frac{1}{2 \times 3}\left[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2\right] \approx 0.58$$

© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (4/9)

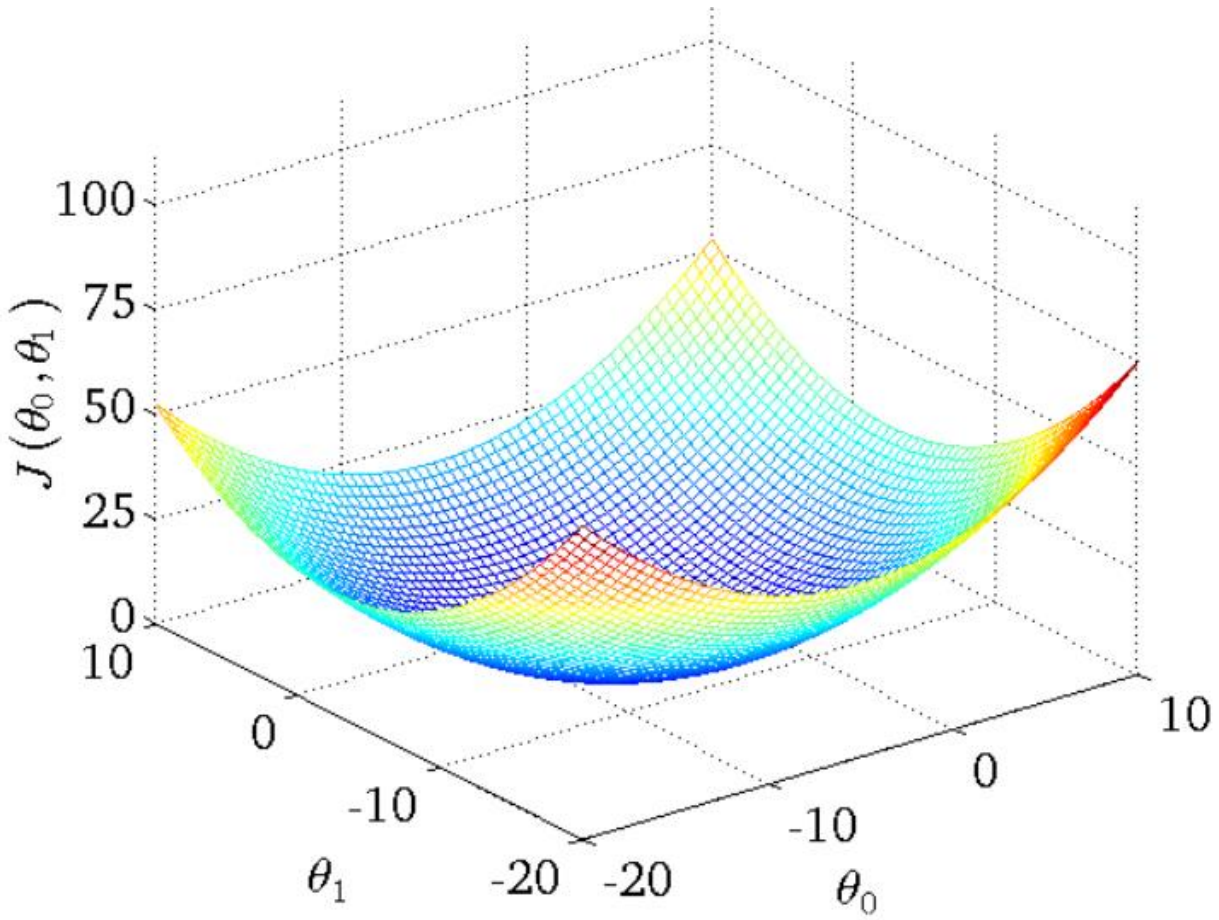
$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right)^2$$

- For insight on J(), let's assume x∈R, $\boldsymbol{\theta} = [\theta_0, \theta_1]$



© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (5/9)

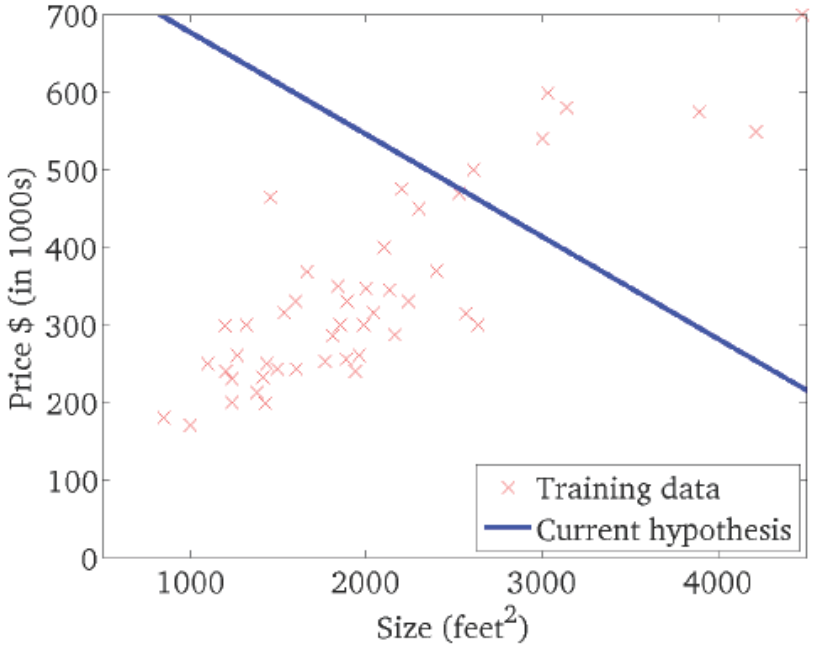


© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (6/9)

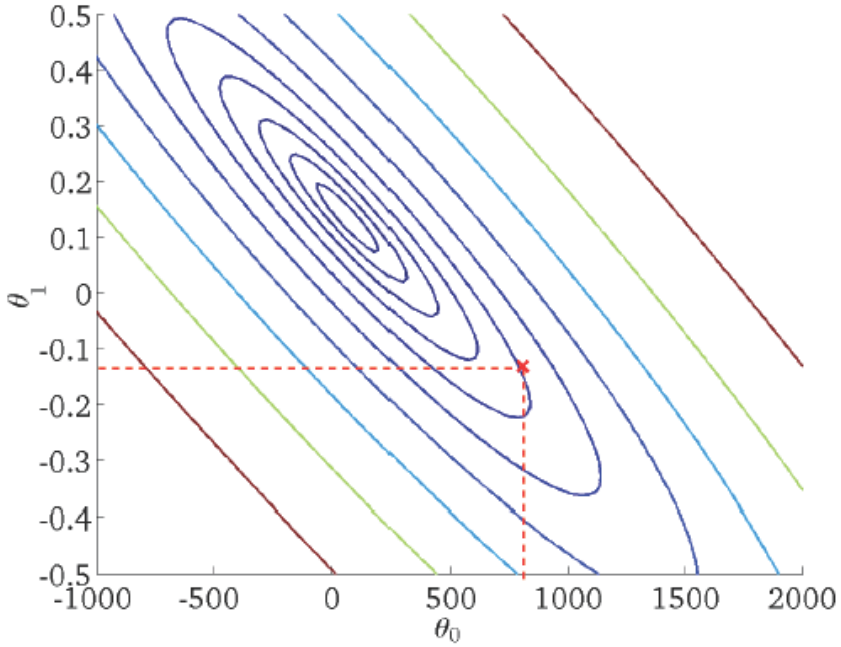
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

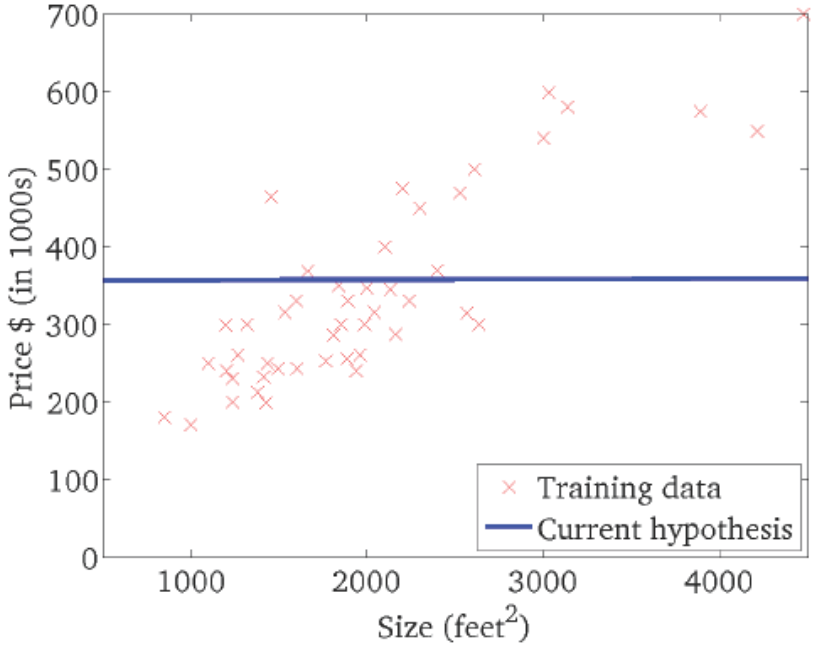


© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (7/9)

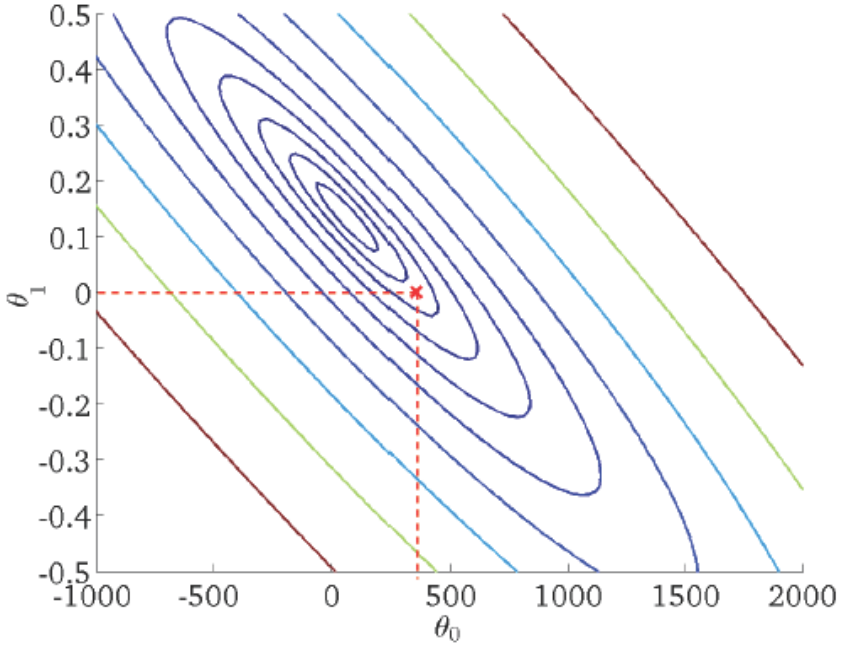
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

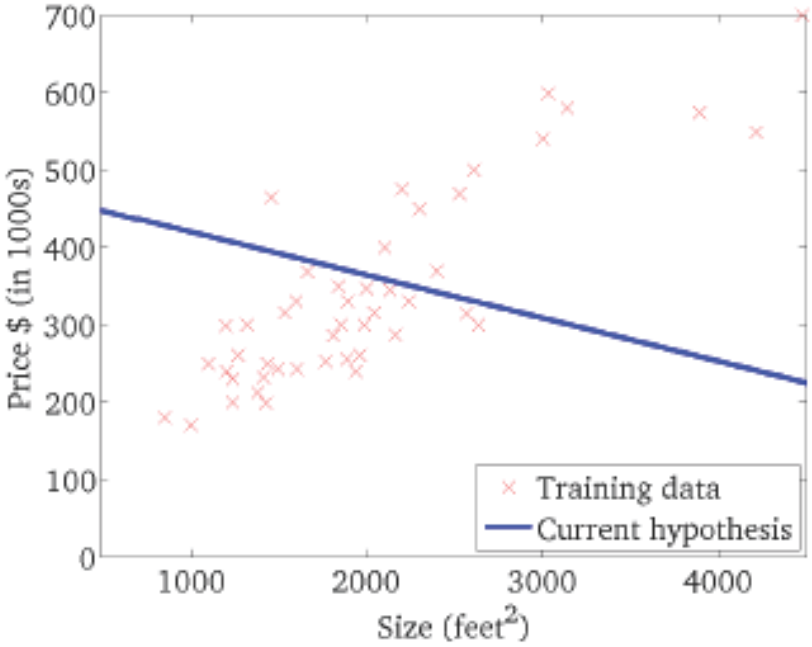


© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (8/9)

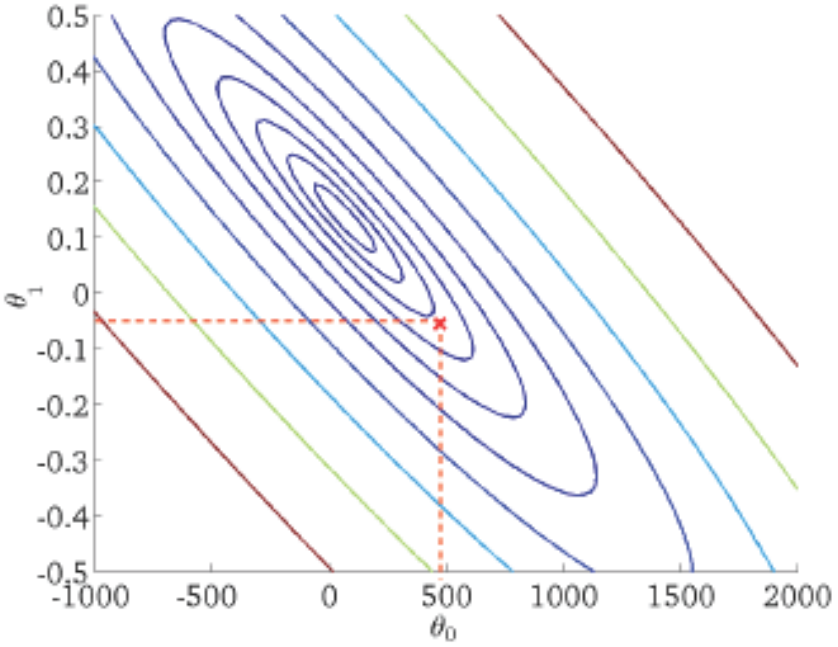
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

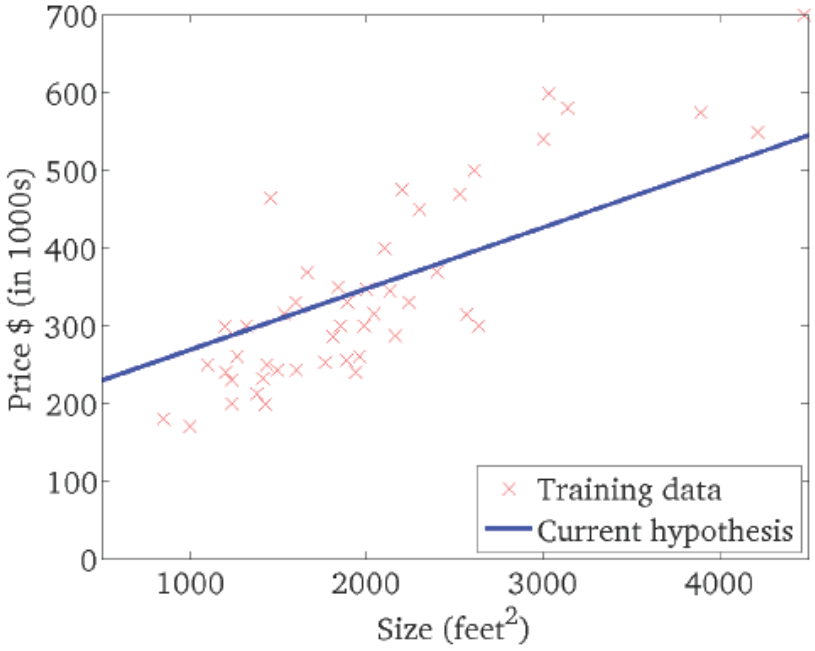


© 2020, UNIVERSITY OF MARYLAND

Intuition Behind Cost Function (9/9)

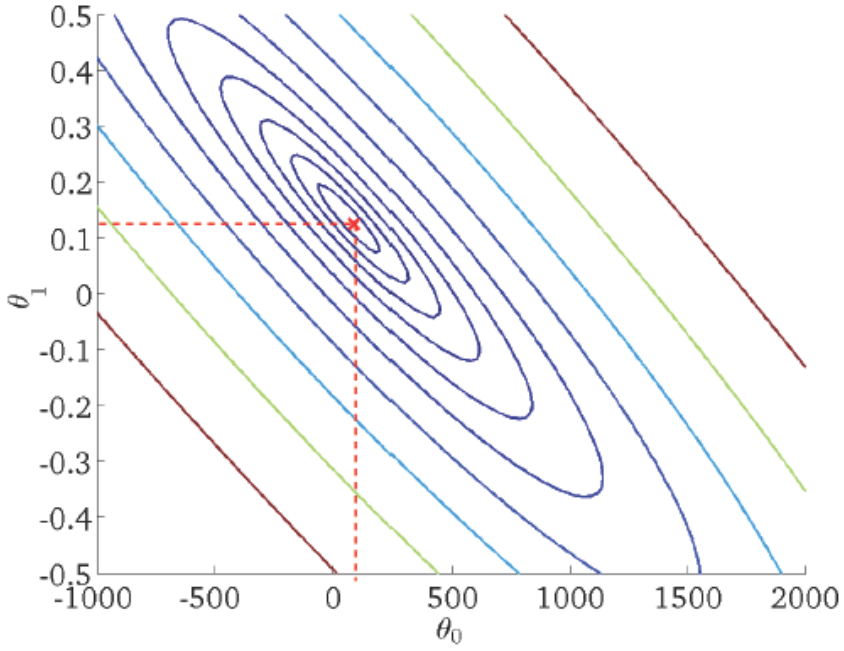
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

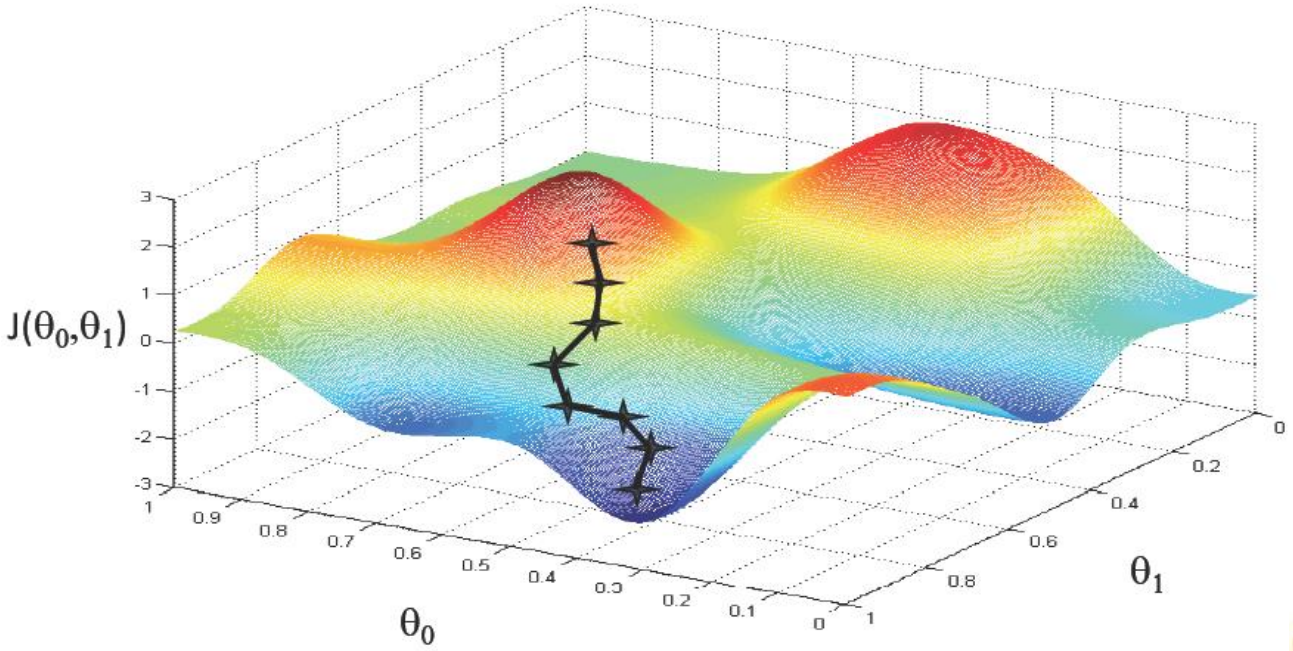
(function of the parameters θ_0, θ_1)



© 2020, UNIVERSITY OF MARYLAND

Basic Search Procedure (1/2)

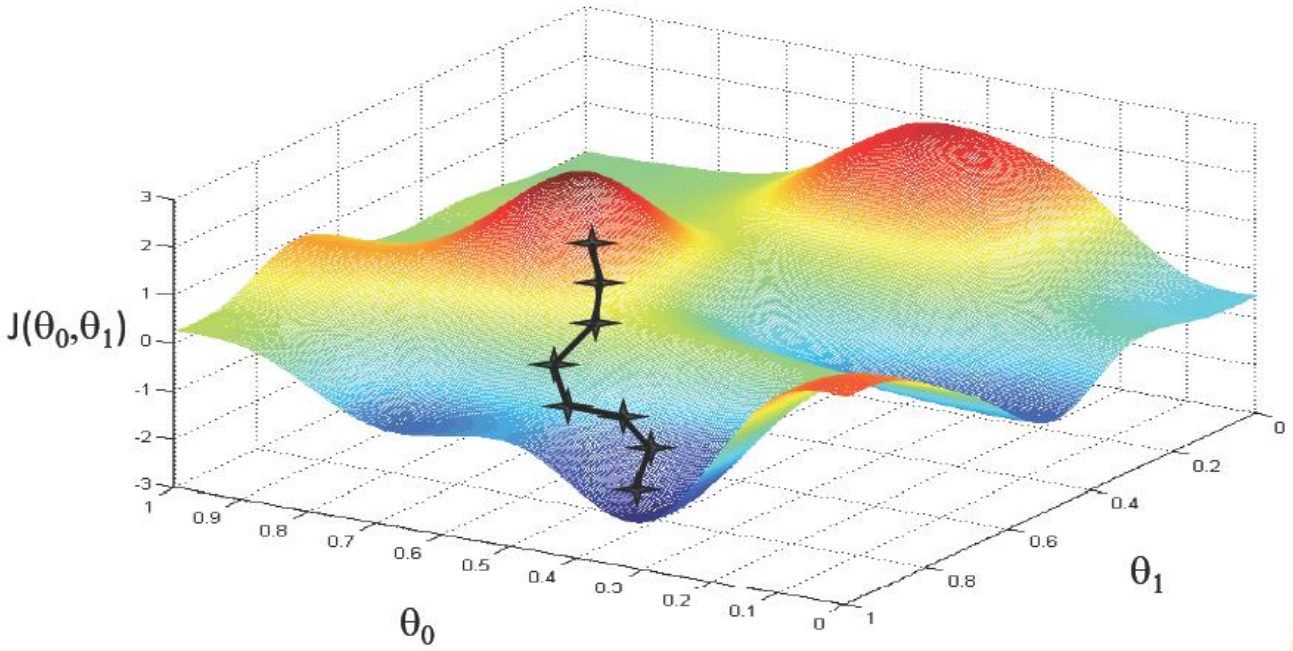
- Choose initial value for $\boldsymbol{\theta}$
- Until we reach a minimum:
  - Choose a new value for $\boldsymbol{\theta}$ to reduce J()



© 2020, UNIVERSITY OF MARYLAND

Basic Search Procedure (2/2)

- Choose initial value for $\boldsymbol{\theta}$
- Until we reach a minimum:
  - Choose a new value for $\boldsymbol{\theta}$ to reduce J()



Since the least squares objective function convex (concave), we do not need to worry about the local minima.

© 2020, UNIVERSITY OF MARYLAND

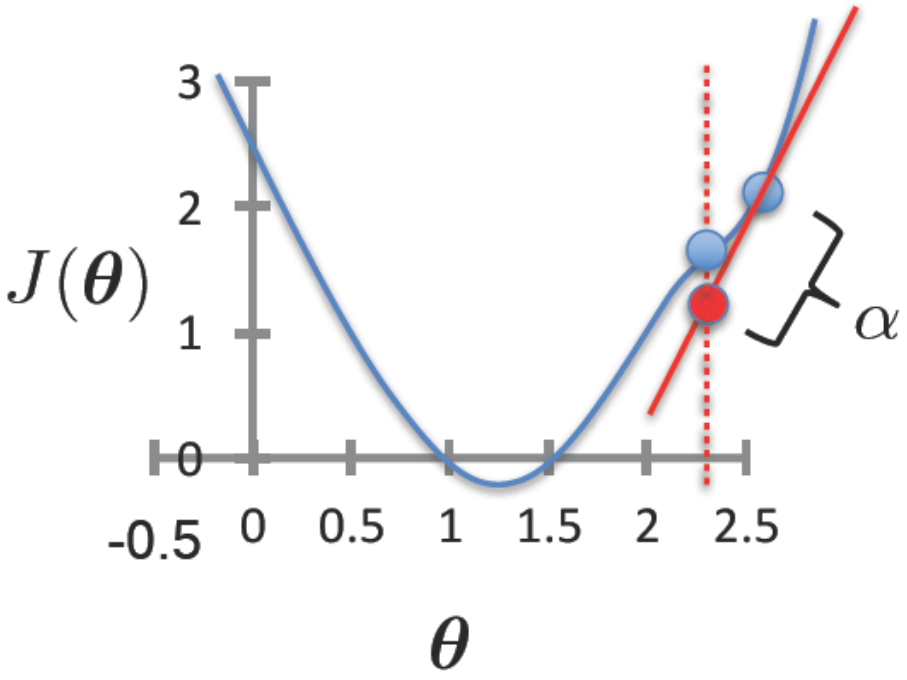
Gradient Descent (1/15)

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$$

simultaneous update for j = 0 … d

learning rate (small)
e.g., α = 0.05



© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (2/15)

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$$

simultaneous update for j = 0 ... d

For Linear Regression:

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) &= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right)^2 \\
&= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right)^2 \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right) \times \frac{\partial}{\partial \theta_j} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right) \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \right) x_j^{(i)}
\end{aligned}$$

© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (3/15)

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)}$$

simultaneous update for $j = 0 \ldots d$

- To achieve simultaneous update
  - At the start of each gradient descent iteration, compute $h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right)$
  - Use this stored value in the update step loop

© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (4/15)

- Assume convergence when $\|\boldsymbol{\theta}_{new} - \boldsymbol{\theta}_{old}\|_2 < \epsilon$

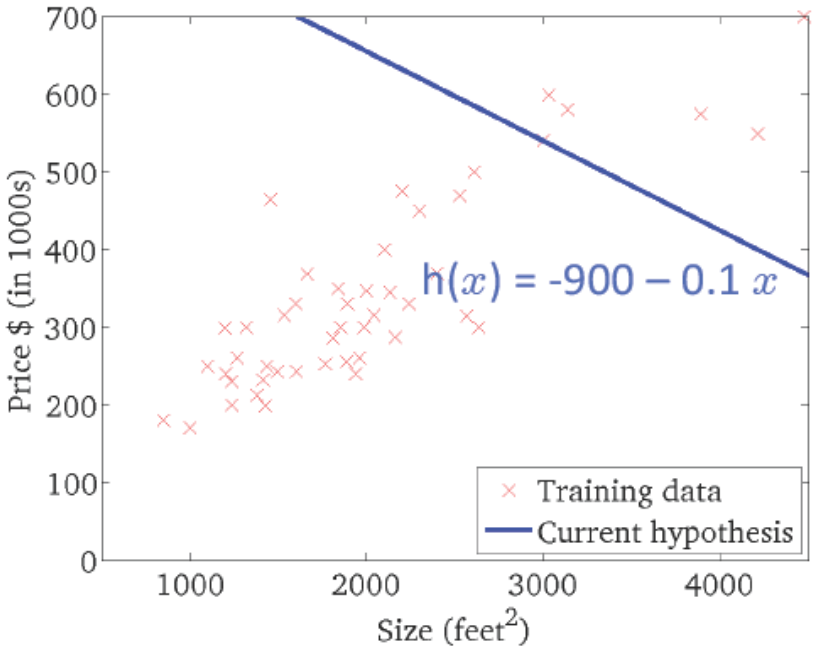
L_2 norm:	$\|\boldsymbol{v}\|_2 = \sqrt{\sum_i v_i^2} = \sqrt{v_1^2 + v_2^2 + \ldots + v_{\|v\|}^2}$

© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (5/15)

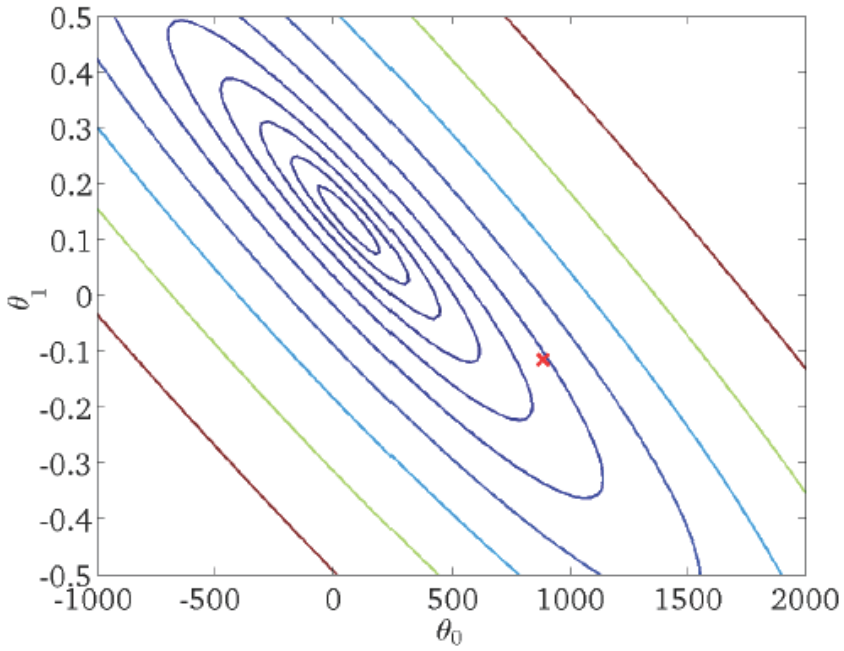
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

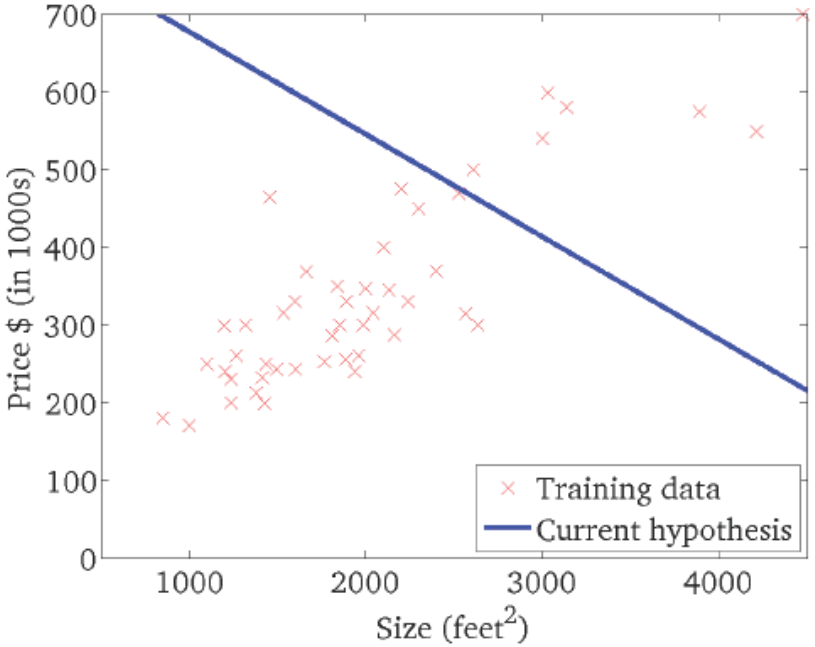


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (6/15)

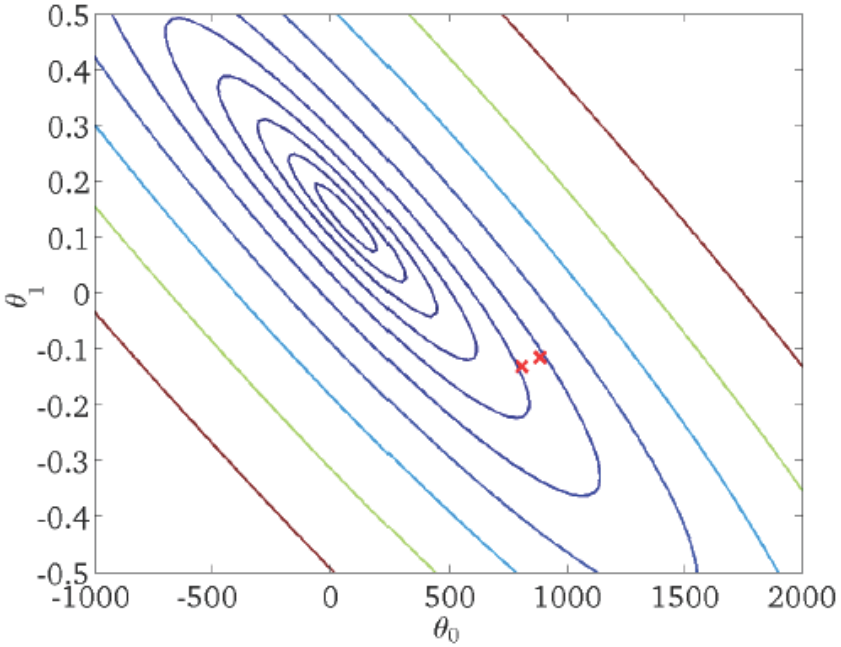
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

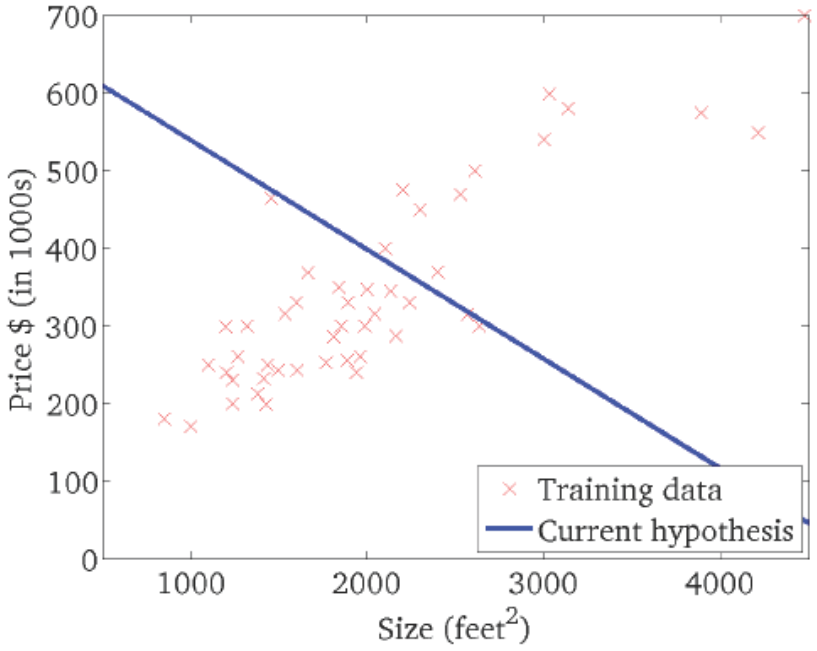


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (7/15)

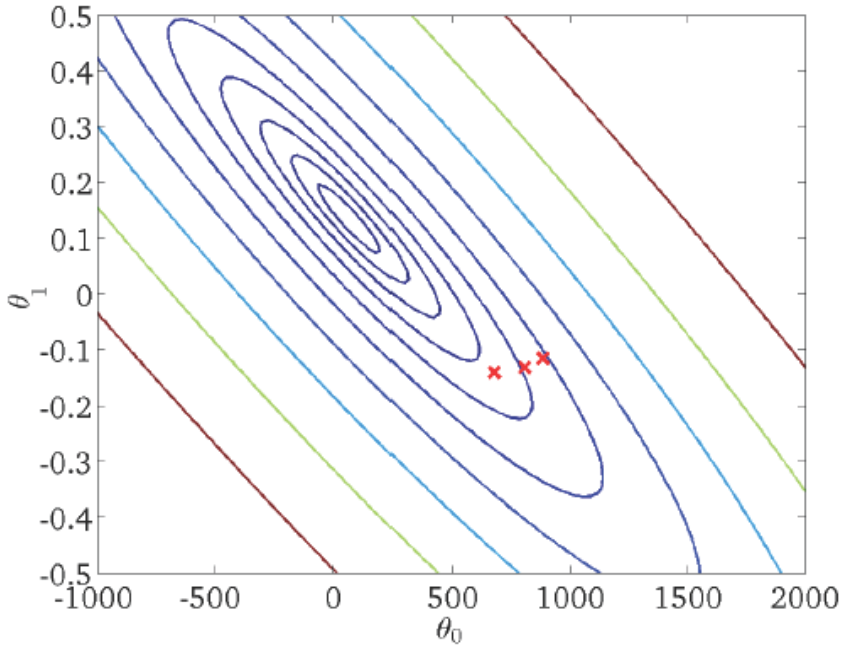
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

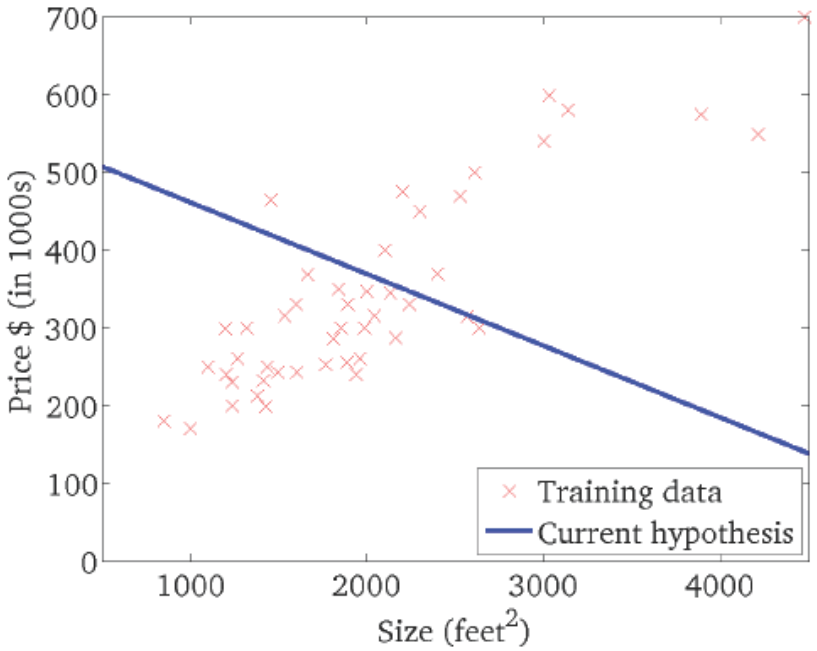


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (8/15)

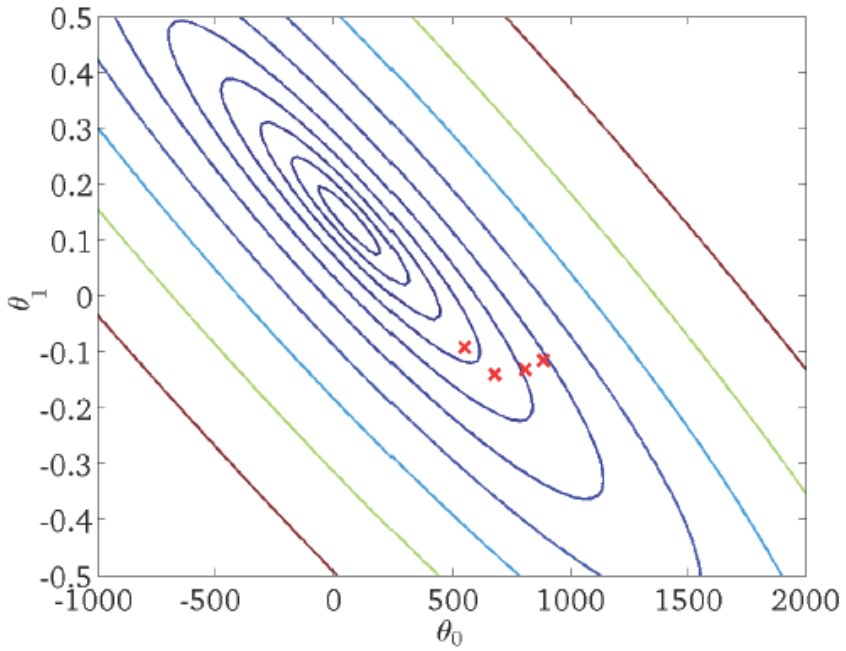
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

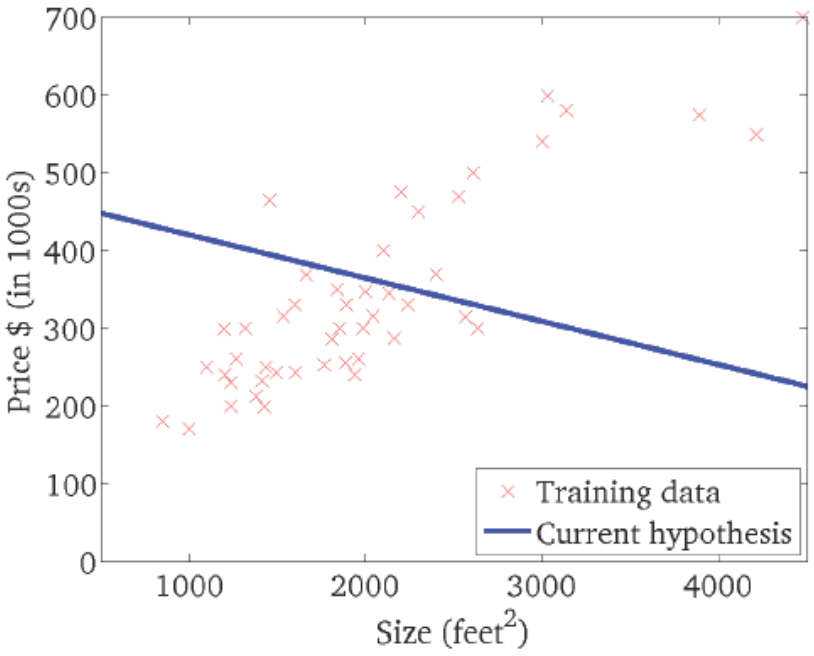


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (9/15)

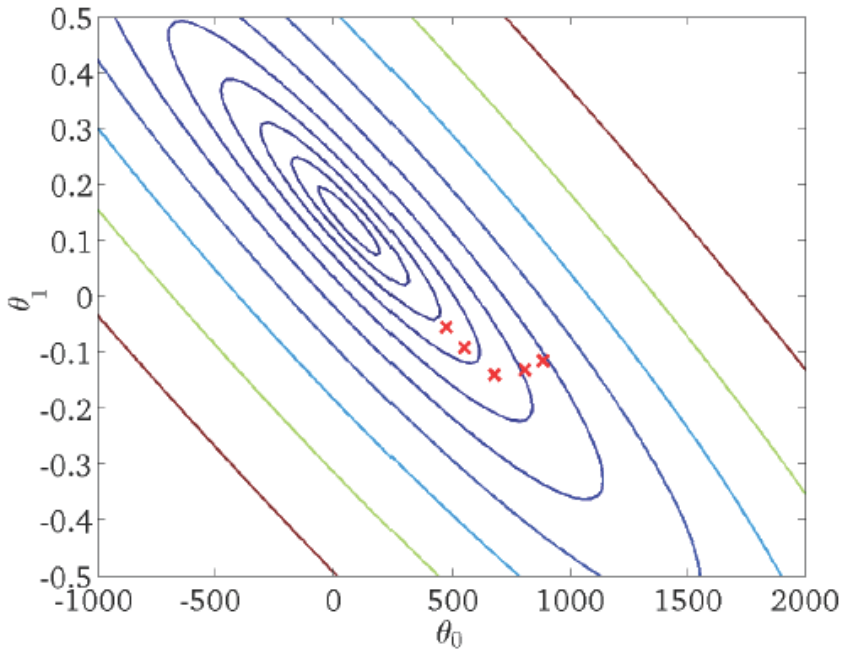
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

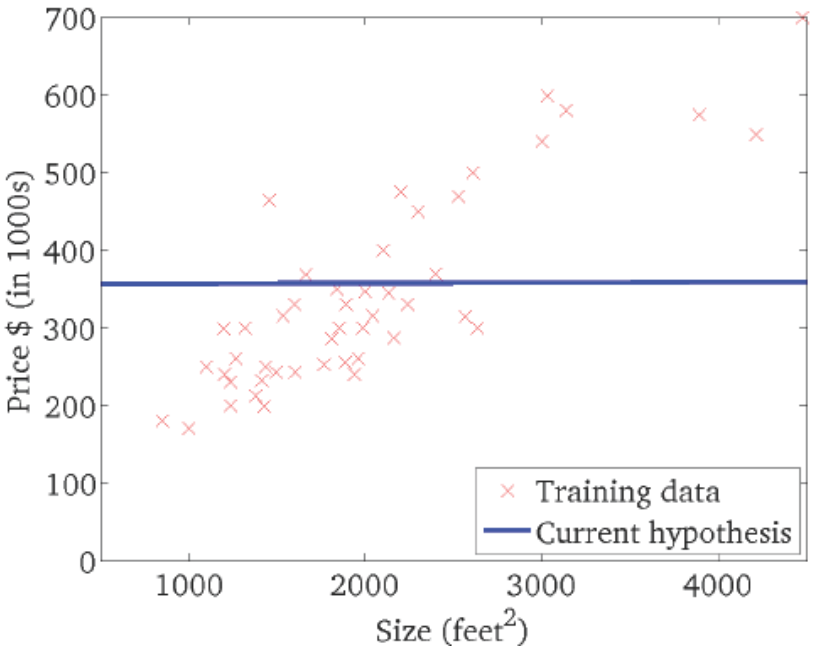


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (10/15)

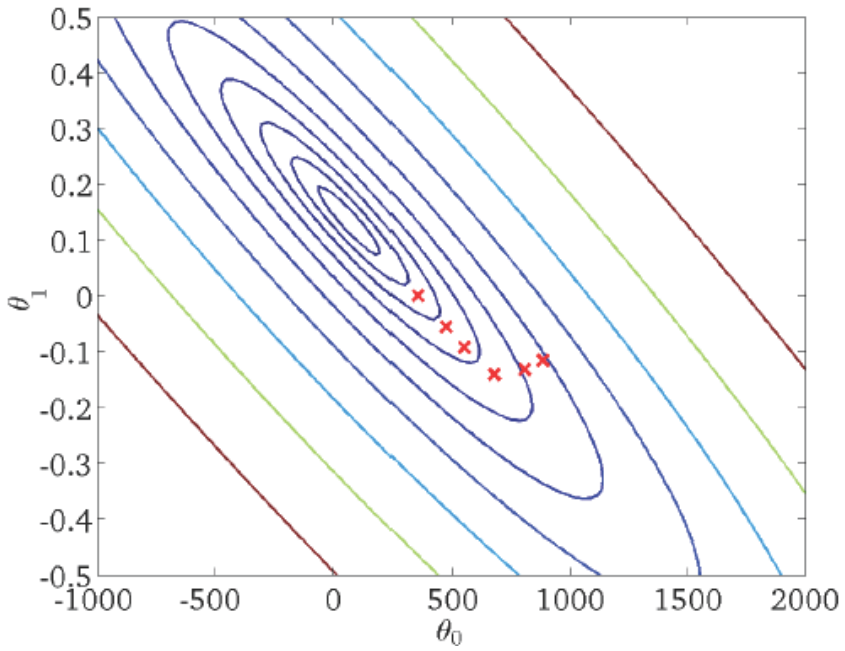
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

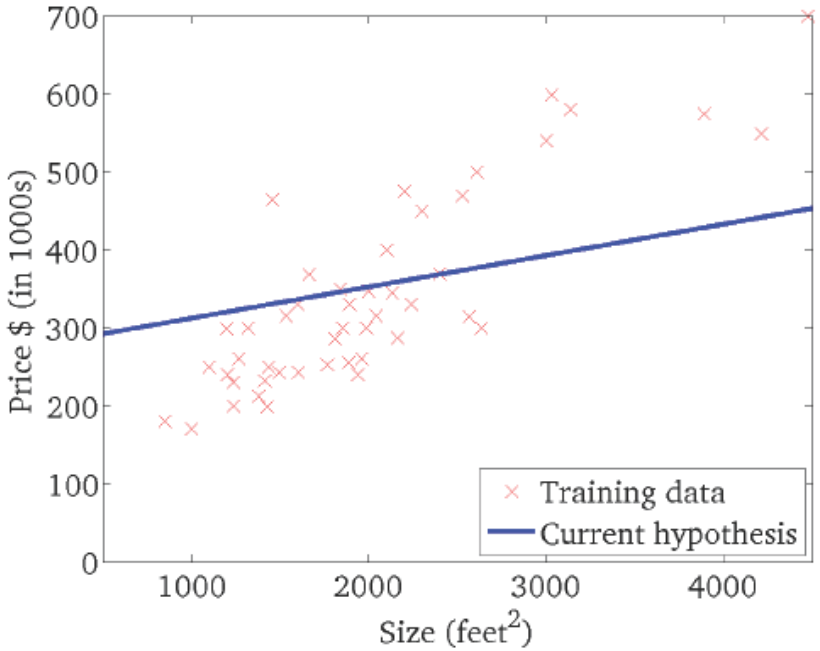


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (11/15)

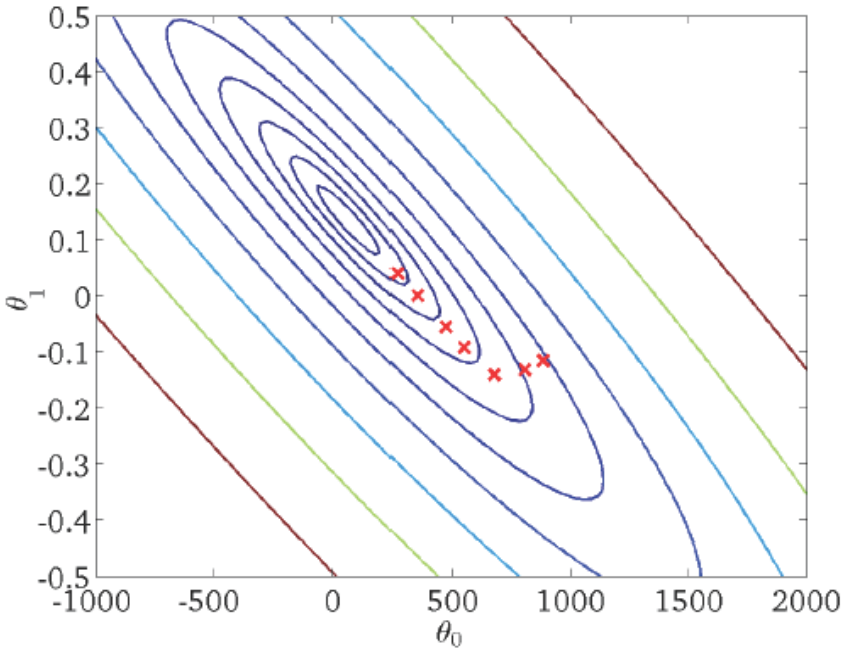
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

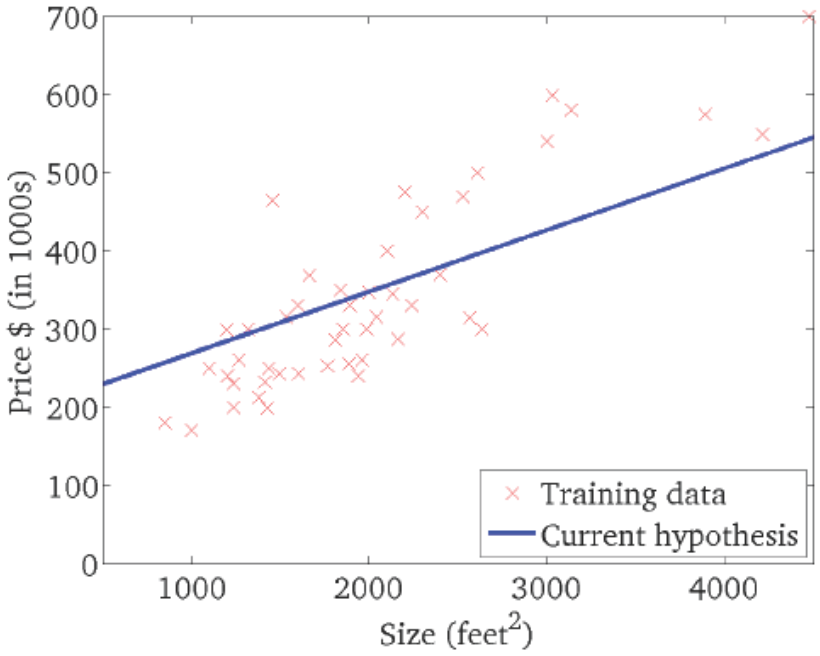


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (12/15)

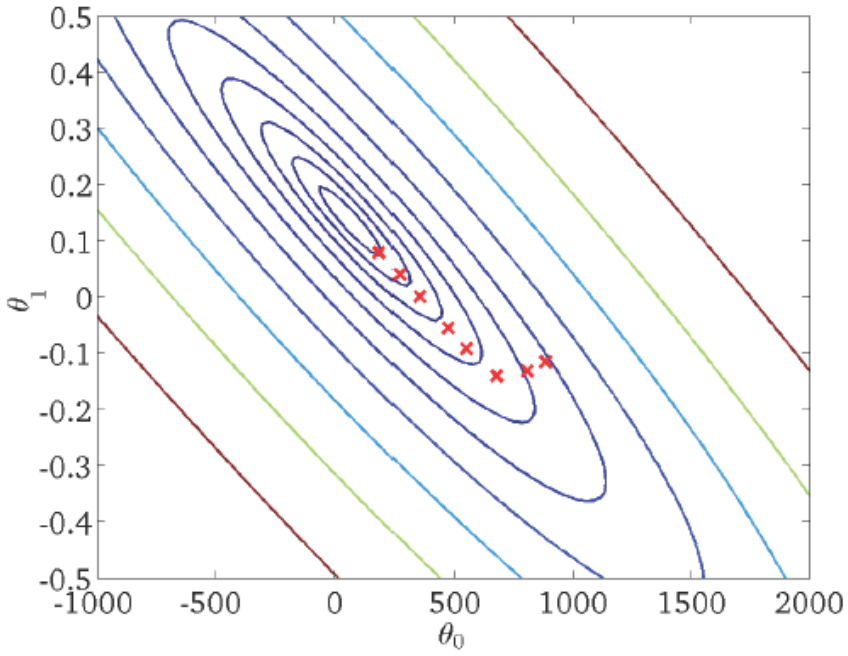
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

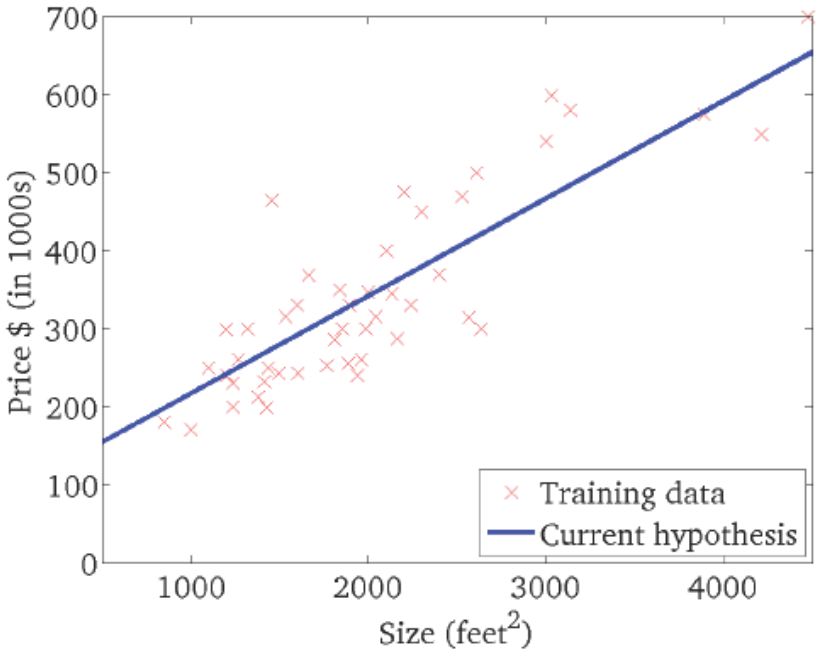


© 2020, UNIVERSITY OF MARYLAND

Gradient Descent (13/15)

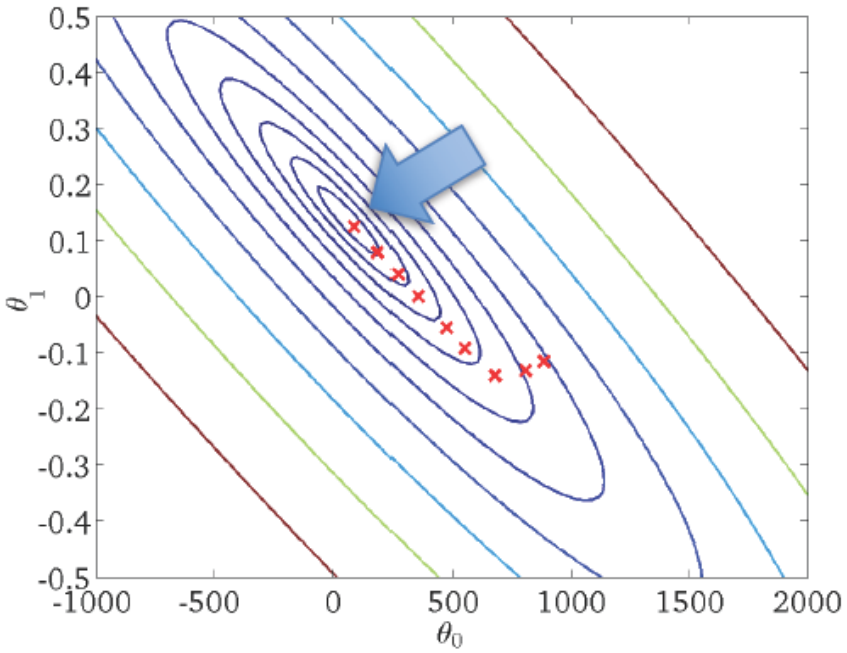
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

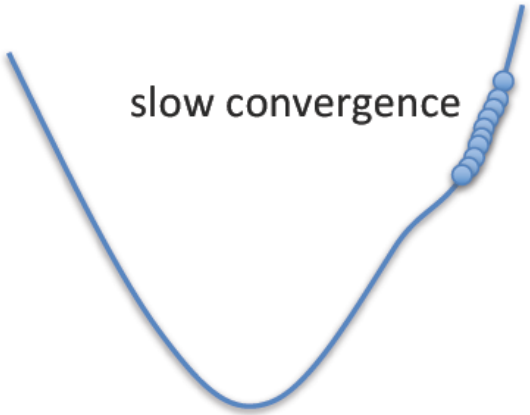


© 2020, UNIVERSITY OF MARYLAND

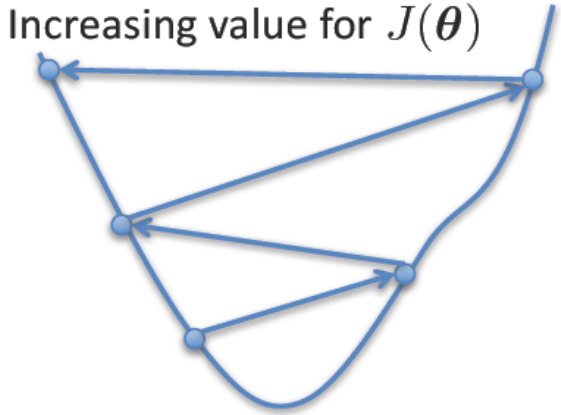
Gradient Descent (14/15)

- Choosing α

α too small



α too large



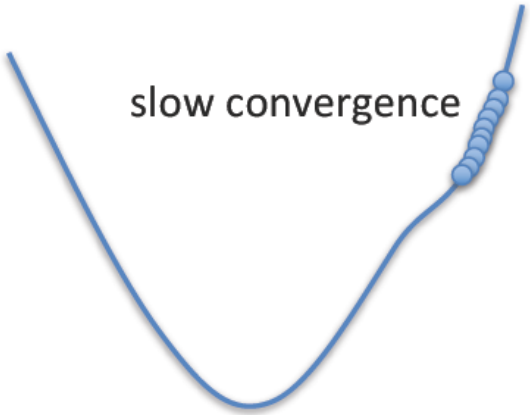
- May overshoot the minimum
- May fail to converge
- May even diverge

© 2020, UNIVERSITY OF MARYLAND

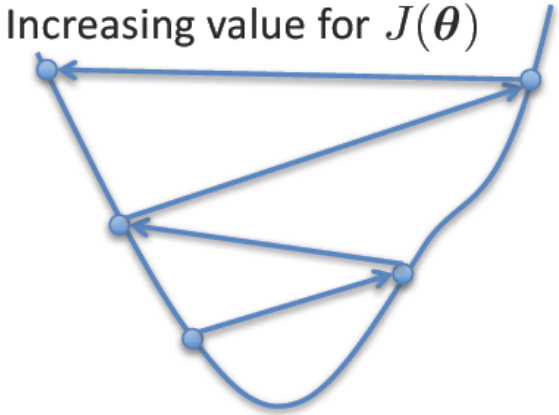
Gradient Descent (15/15)

- Choosing α

α too small



α too large



- May overshoot the minimum
- May fail to converge
- May even diverge

To see if gradient descent is working, print out $J(\theta)$ each iteration
- The value should decrease at each iteration
- If it doesn't, adjust α

© 2020, UNIVERSITY OF MARYLAND

```python
from numpy import asarray
from numpy import arange
from numpy.random import rand
from matplotlib import pyplot

def objective(x):
    return x**2.0

def derivative(x):
    return x * 2.0

def gradient_descent(objective, derivative, bounds, n_iter, step_size):
    solutions, scores = list(), list()
    solution = bounds[:, 0] + rand(len(bounds)) * (bounds[:, 1] - bounds[:, 0])
    for i in range(n_iter):
        gradient = derivative(solution)
        solution = solution - step_size * gradient
        solution_eval = objective(solution)
        solutions.append(solution)
        scores.append(solution_eval)
        print('>%d f(%s) = %.5f' % (i, solution, solution_eval))
    return [solutions, scores]

bounds = asarray([[-1.0, 1.0]])
n_iter = 30
step_size = 0.1
solutions, scores = gradient_descent(objective, derivative, bounds, n_iter, step_size)
inputs = arange(bounds[0,0], bounds[0,1]+0.1, 0.1)
results = objective(inputs)
pyplot.plot(inputs, results)
pyplot.plot(solutions, scores, '.-', color='red')
pyplot.show()
```


© 2020, UNIVERSITY OF MARYLAND

Issue of Gradient Descent (1/2)

- Fit via gradient descent:

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)}$$

simultaneous update for $j = 0 \ldots d$

- What is the computational complexity in terms of n?

© 2020, UNIVERSITY OF MARYLAND

Issue of Gradient Descent (2/2)

- Fit via gradient descent:

- Initialize $\boldsymbol{\theta}$
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}} \left(\boldsymbol{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)}$$

simultaneous update for $j = 0 \ldots d$

- Issue:
  - What is the computational complexity in terms of n?
  - Too slow!

© 2020, UNIVERSITY OF MARYLAND

Stochastic Gradient Descent

- Batch gradient descent

Initialize θ

Repeat {

$$\theta_j \leftarrow \theta_j - \alpha \underbrace{\frac{1}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}\left(\mathbf{x}_i\right) - y_i\right) x_{ij}}_{\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})} \qquad \text{for } j = 0 \ldots d$$

}

- Stochastic gradient descent

Initialize θ

Randomly shuffle dataset

Repeat { (Typically 1 – 10x)

For $i = 1 \ldots n$, *do*

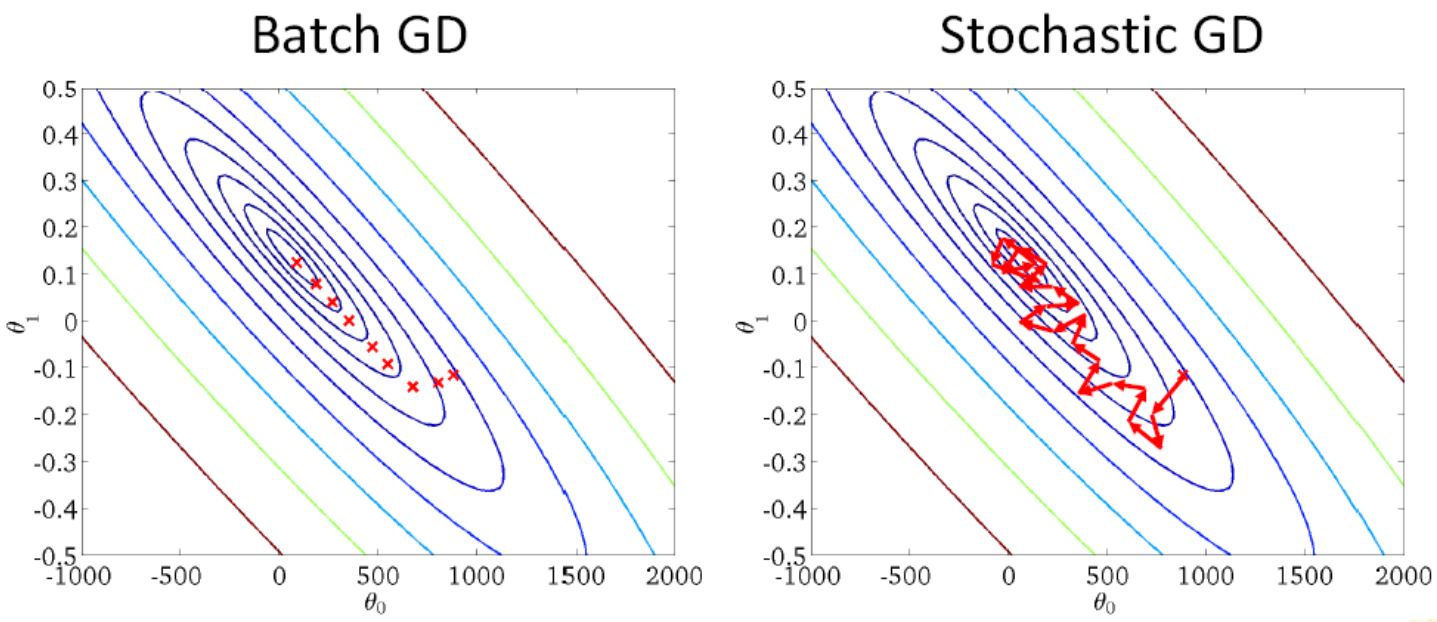
$$\theta_j \leftarrow \theta_j - \alpha \underbrace{\left(h_{\boldsymbol{\theta}}\left(\mathbf{x}_i\right) - y_i\right) x_{ij}}_{\frac{\partial}{\partial \theta_j} \mathrm{cost}_{\boldsymbol{\theta}}(\mathbf{x}_i, y_i)} \qquad \text{for } j = 0 \ldots d$$

}

© 2020, UNIVERSITY OF MARYLAND

Batch vs. Stochastic Gradient Descent

- Learning rate α is typically held constant
- Can slowly decrease α over time to force learning converge



$$\text{e.g.,} \quad \alpha_t = \frac{\text{constant1}}{\text{iterationNumber} + \text{constant2}}$$

© 2020, UNIVERSITY OF MARYLAND

New Stochastic Gradient Descent (1/5)

- Mini-batch gradient descent
  - Use a small set of samples in each iteration

Repeat{

For i=1:m{

$$\theta_j := \theta_j - \alpha \frac{1}{b} \sum_{k=i}^{i+b-1} (h_\theta(x^{(k)}) - y^{(k)}) x_j^{(k)}$$

(for j=0:n)

i +=10;

}

© 2020, UNIVERSITY OF MARYLAND

New Stochastic Gradient Descent (2/5)

- So far, we have considered:
  - A constant learning rate α
  - A time-dependent learning rate αt via a pre-set formula
- **Adagrad** adjusts the learning rate based on **historical information**
  - Frequently occurring features in the gradients get small learning rates and infrequent features get higher ones
  - Key idea: “learn slowly” from frequent features but “pay attention” to rare but informative features

© 2020, UNIVERSITY OF MARYLAND

New Stochastic Gradient Descent (3/5)

- Define a **per-feature learning rate** for feature j as:

$$\alpha_{t,j} = \frac{\alpha}{\sqrt{G_{t,j}}} \quad \text{where} \quad G_{t,j} = \sum_{k=1}^{t} \underbrace{g_{k,j}^2}_{\frac{\partial}{\partial \theta_j} \mathrm{cost}_{\boldsymbol{\theta}}(\mathbf{x}_k, y_k)}$$

  - $G_{t,j}$ is the sum of squares of gradients of feature j through t

© 2020, UNIVERSITY OF MARYLAND

New Stochastic Gradient Descent (4/5)

Adagrad per-feature learning rate

$$\alpha_{t,j} = \frac{\alpha}{\sqrt{G_{t,j}}} \quad \text{where} \quad G_{t,j} = \sum_{k=1}^{t} g_{k,j}^2$$

- Adagrad changes the update rule for SGD at time t from

$$\theta_j \leftarrow \theta_j - \alpha g_{t,j}$$

to

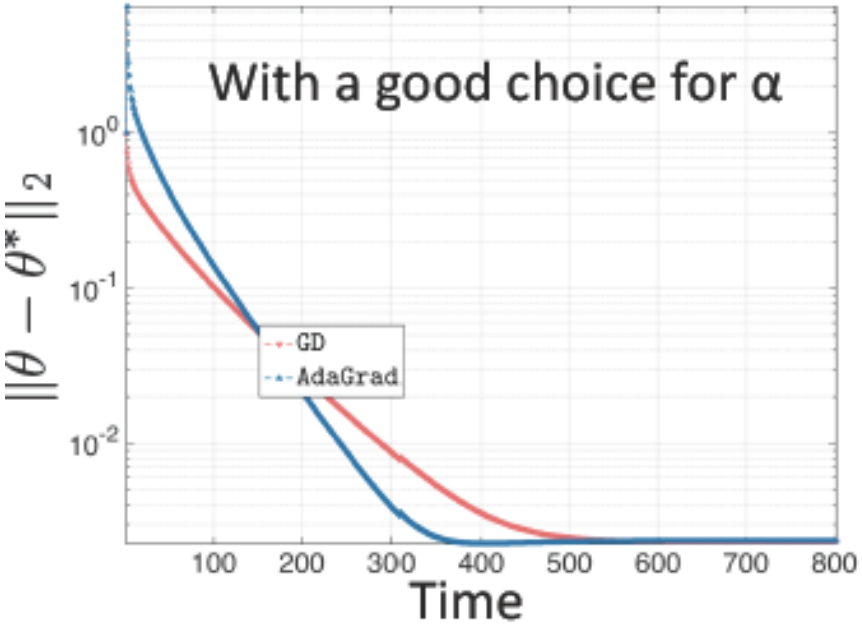
$$\theta_j \leftarrow \theta_j - \frac{\alpha}{\sqrt{G_{t,j}} + \zeta} g_{t,j}$$

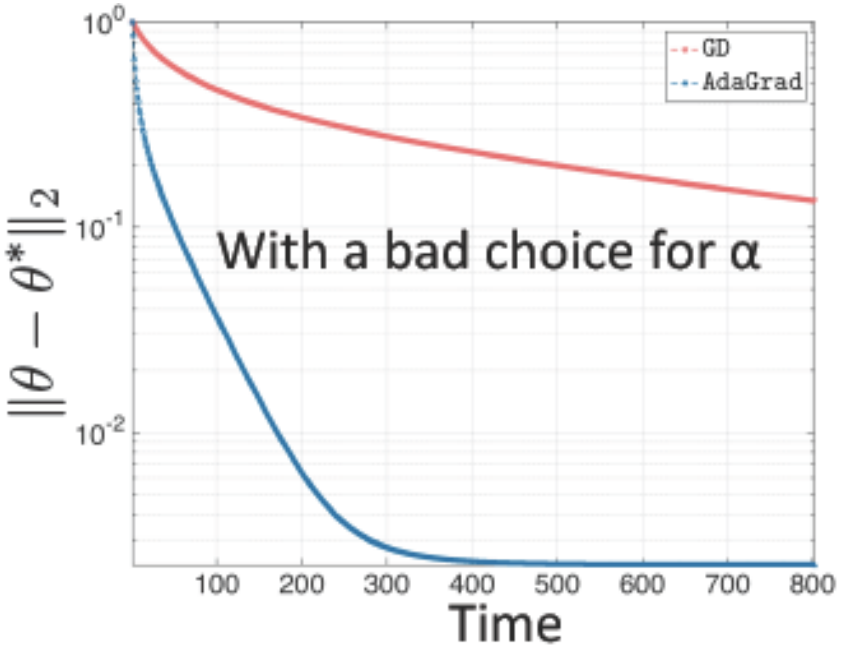
In practice, we add a small constant $\zeta > 0$ to prevent dividing by zero errors

© 2020, UNIVERSITY OF MARYLAND

New Stochastic Gradient Descent (5/5)

- Adagrad converges quickly:





© 2020, UNIVERSITY OF MARYLAND

Linear Basis Function Models (1/4)

- Generally, $h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j \underbrace{\phi_j(\boldsymbol{x})}_{\text{basis function}}$

- Typically, $\phi_0(\boldsymbol{x}) = 1$ so that θ_0 acts as a bias
- In the simplest case, we use linear basis functions:

$$\phi_j(\boldsymbol{x}) = x_j$$

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_d x_d = \sum_{j=0}^{d} \theta_j x_j$$

Assume $x_0 = 1$

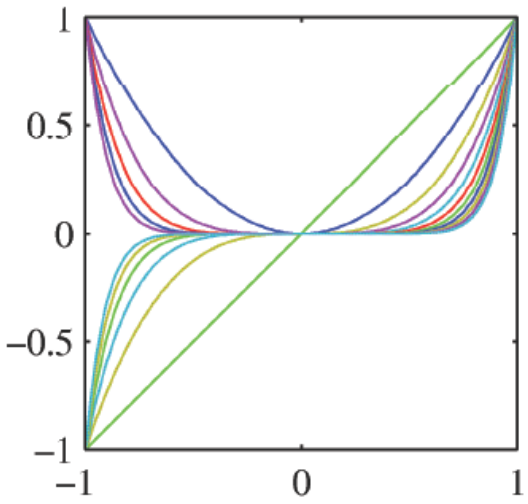
© 2020, UNIVERSITY OF MARYLAND

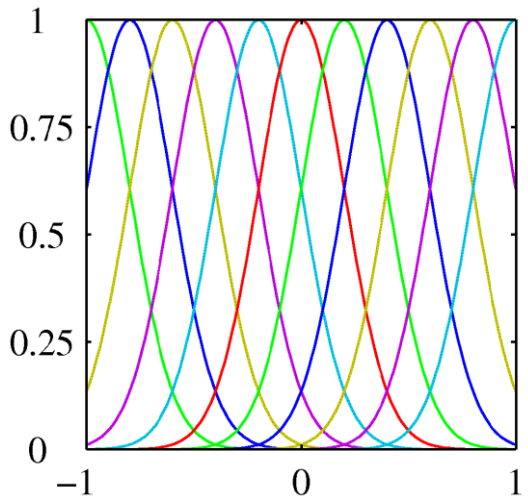
Linear Basis Function Models (2/4)

- Polynomial basis functions:

$$\phi_j(x) = x^j$$

  - These are global; a small change in x affects all basis functions





- Gaussian basis functions:

$$\phi_j(x) = \exp\left\{-\frac{(x-\mu_j)^2}{2s^2}\right\}$$

  - These are local; a small change in x only affect nearby basis functions. μj and s control location and scale (width).

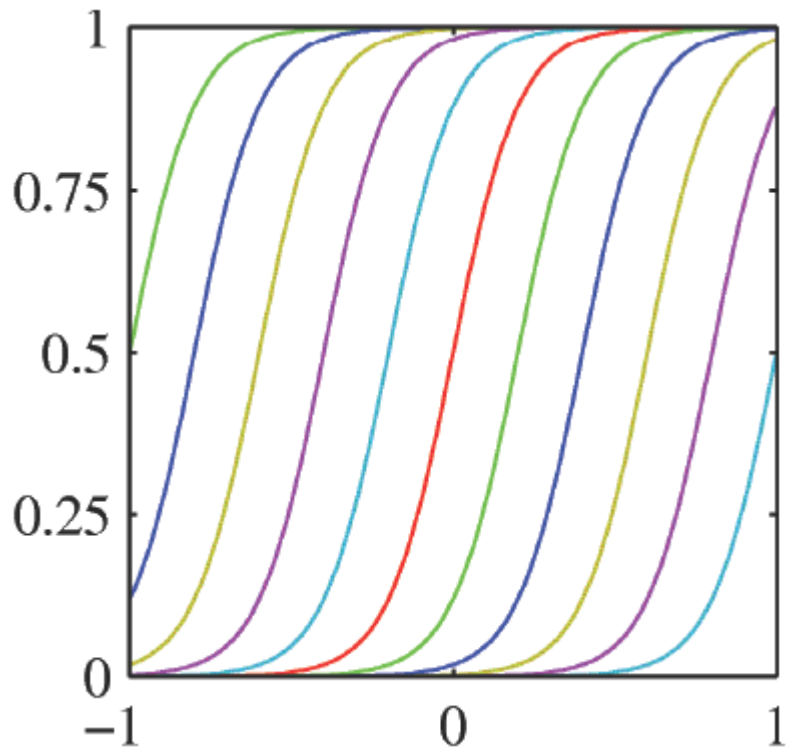
© 2020, UNIVERSITY OF MARYLAND

Linear Basis Function Models (3/4)

- Sigmoidal basis functions

$$\phi_j(x) = \sigma\left(\frac{x - \mu_j}{s}\right)$$

where $\sigma(a) = \dfrac{1}{1 + \exp(-a)}$

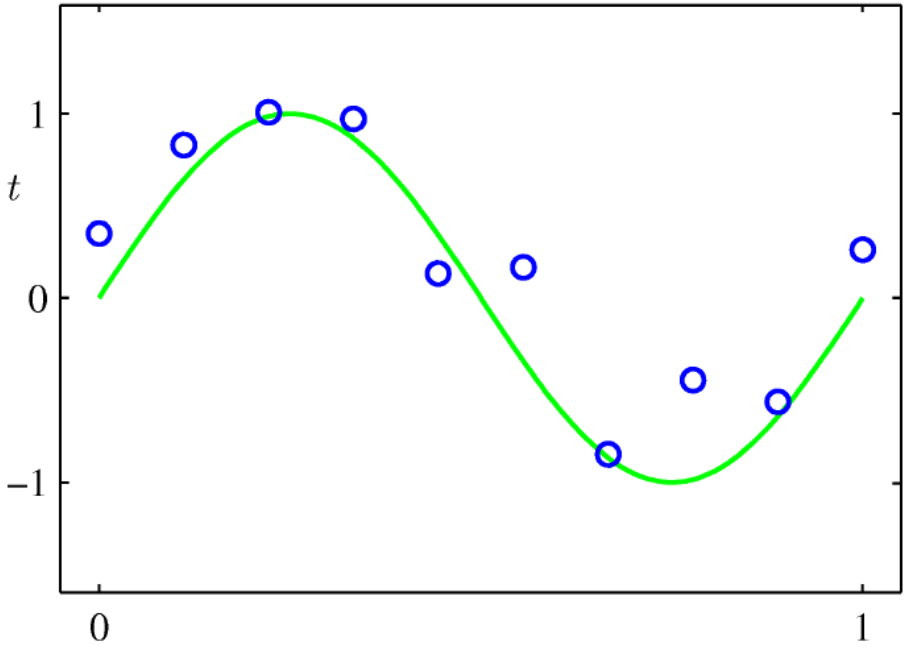


These are also local; a small change in x only affects nearby basis functions. μj and s control location and scale (slope).

© 2020, UNIVERSITY OF MARYLAND

Linear Basis Function Models (4/4)

- Example: Fitting a polynomial curve with a linear model



$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_p x^p = \sum_{j=0}^{p} \theta_j x^j$$

© 2020, UNIVERSITY OF MARYLAND

Summary (1/2)

- Basic linear model: $h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j x_j$

- Generalized linear model: $h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j \phi_j(\boldsymbol{x})$

© 2020, UNIVERSITY OF MARYLAND

Summary (2/2)

- Once we have replaced the data by the outputs of the basis functions, fitting the generalized model is exactly the same problem as fitting the basic model
  - Unless we use the kernel trick – more on that when we cover support vector machines
  - Therefore, there is no point in cluttering the math with basis functions

© 2020, UNIVERSITY OF MARYLAND